โครงการอบรมโอลิมปิกวิชาการ ค่าย 2 สาขาคอมพิวเตอร์
วันที่ 27 มี.ค. – 10 เม.ย. 2569

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเชียงใหม่

Binary Trees and Binary Search Trees

วันที่ 30 มีนาคม 2569 (บ่าย)

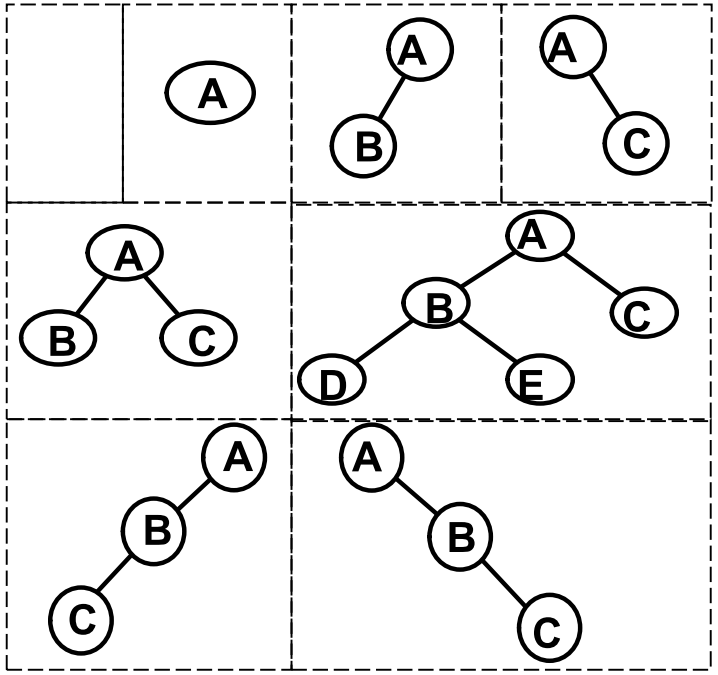
ผู้ช่วยศาสตราจารย์เบญจมาศ ปัญญางาม

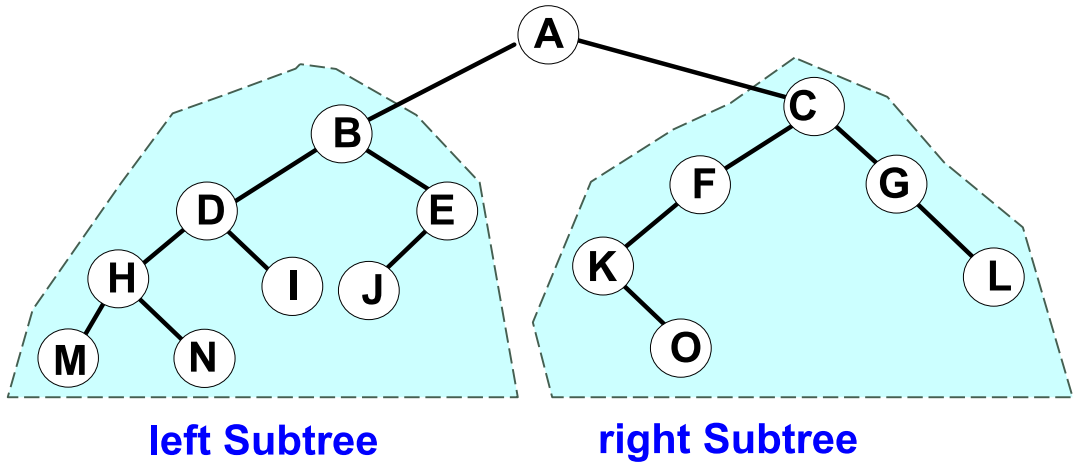
Outline

- Binary Tree
  - ✓ Tree Traversal
- Binary Search Tree
- Tree Construction
- Binary Tree Application

Binary Tree

- แต่ละ node มี child ได้ไม่เกิน 2 node เรียก left child และ right child

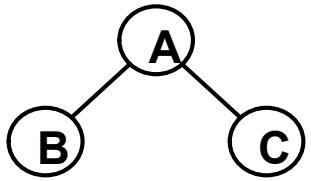


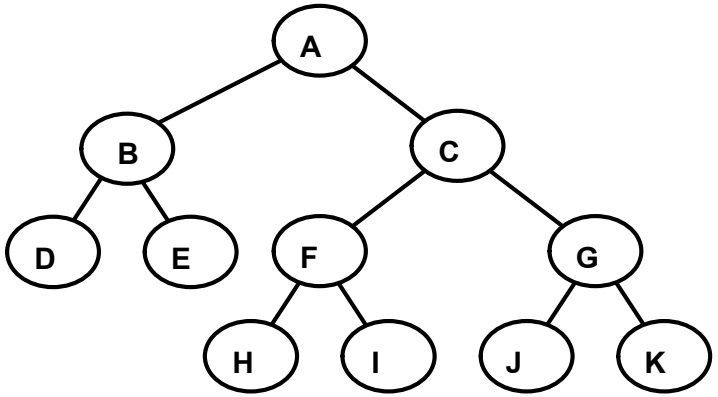


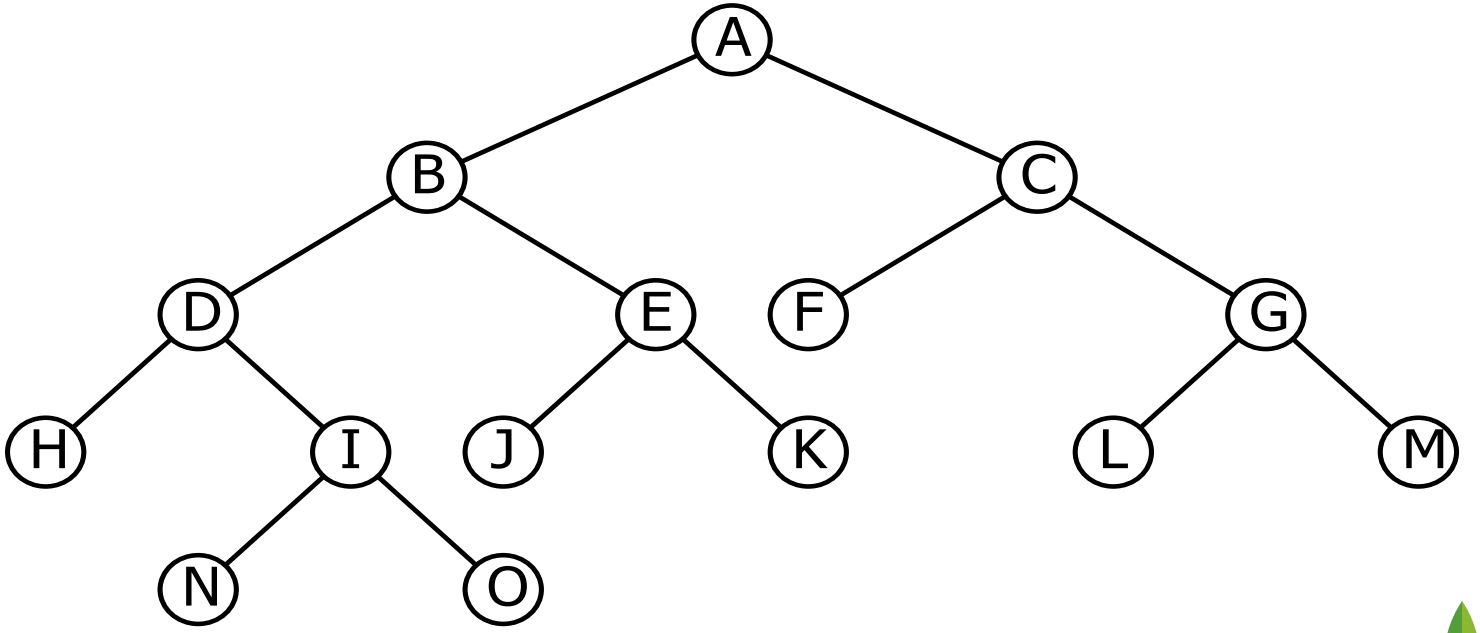
REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Full Binary Tree

- ทุกโหนด (ยกเว้น leaves) มี Degree เท่ากับ 2



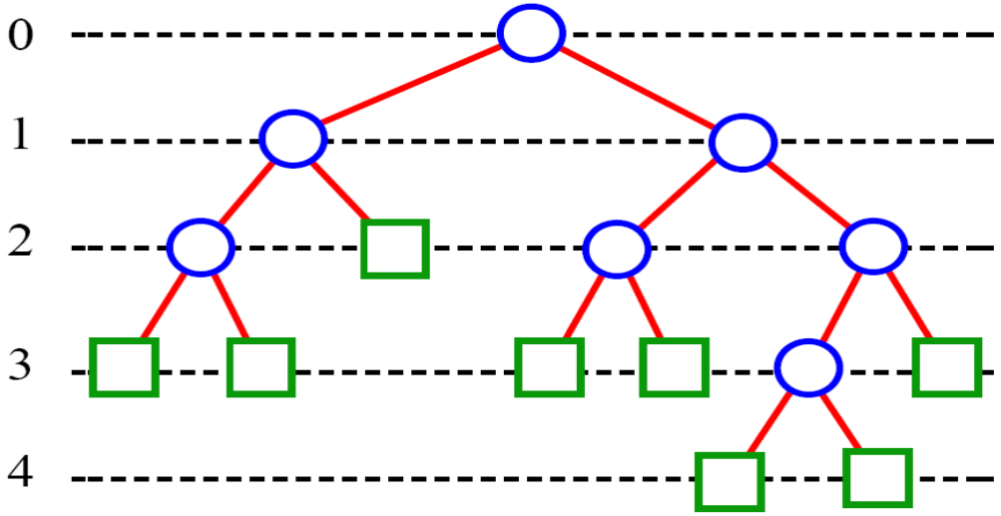




REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Properties of Binary Trees

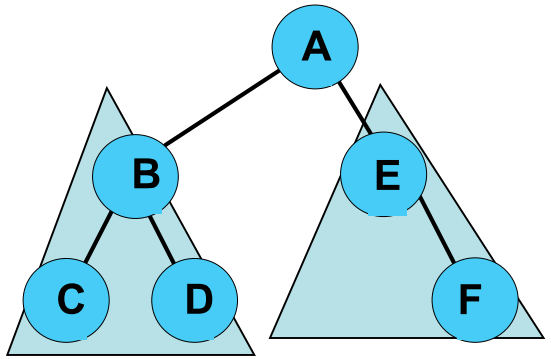
- จำนวน nodes ใน level $i \leq 2^i$
- จำนวน external nodes = (จำนวน nodes + 1) / 2
- จำนวน external nodes $\leq 2^{height}$
- height $\geq \log_2$ (จำนวน external nodes)



REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

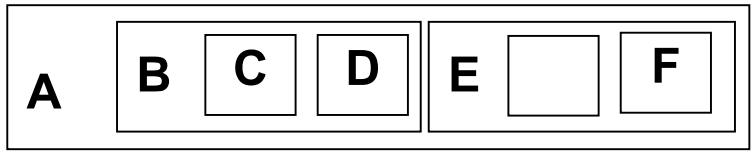
Binary Tree : Traversal

- Example



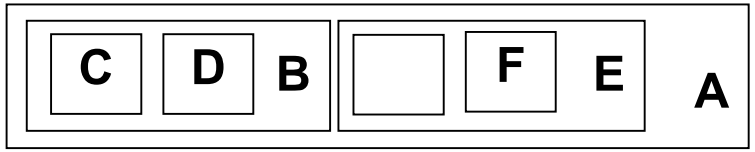
แบบ preOrder

R T_L T_R



แบบ Postorder

T_L T_R R



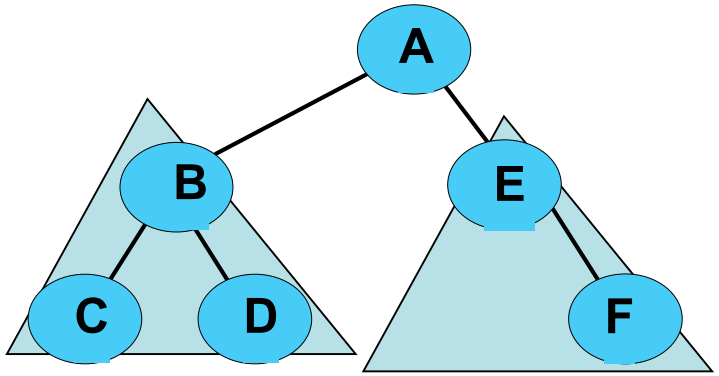

```
preOrder(p) {
  Print item in node p
  if hasLeft (p)
      preOrder (leftchild (p))
  if hasRight (p)
      preOrder (rightchild (p))
}
```

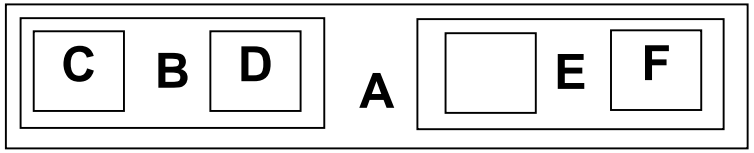
```
postOrder(p) {
  if hasLeft (p)
      postOrder (leftchild (p))
  if hasRight (p)
      postOrder (rightchild (p))
  Print item in node p
}
```

Binary Tree : Traversal

- แบบ Inorder

T_L **R** T_R

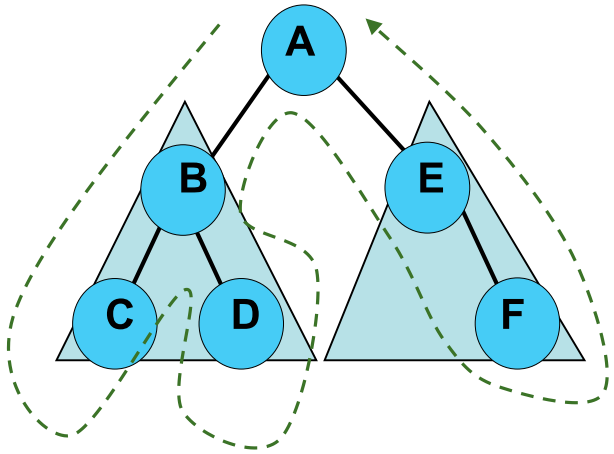




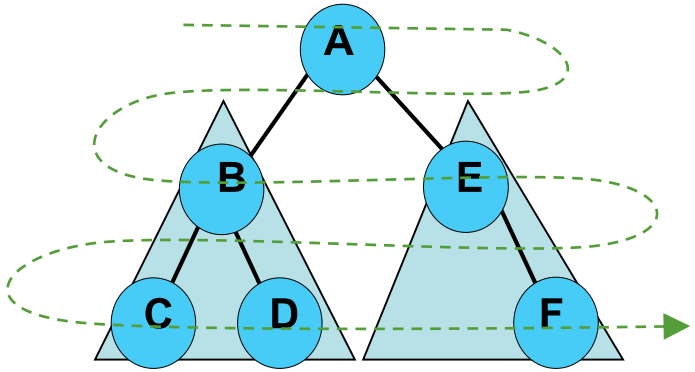

```
inOrder(p) {
  if hasLeft (p)
      inOrder (leftchild (p))
  Print item in node p
  if hasRight (p)
      inOrder (rightchild (p))
}
```

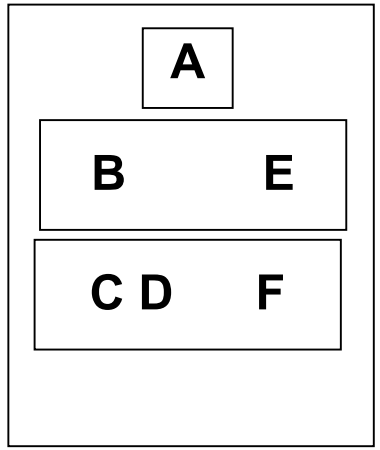
Binary Tree Traversal : Traversal

- **Depth-first Search**
  - ✓ preOrder
  - ✓ postOrder
  - ✓ inOrder



- **Breadth-first Search**





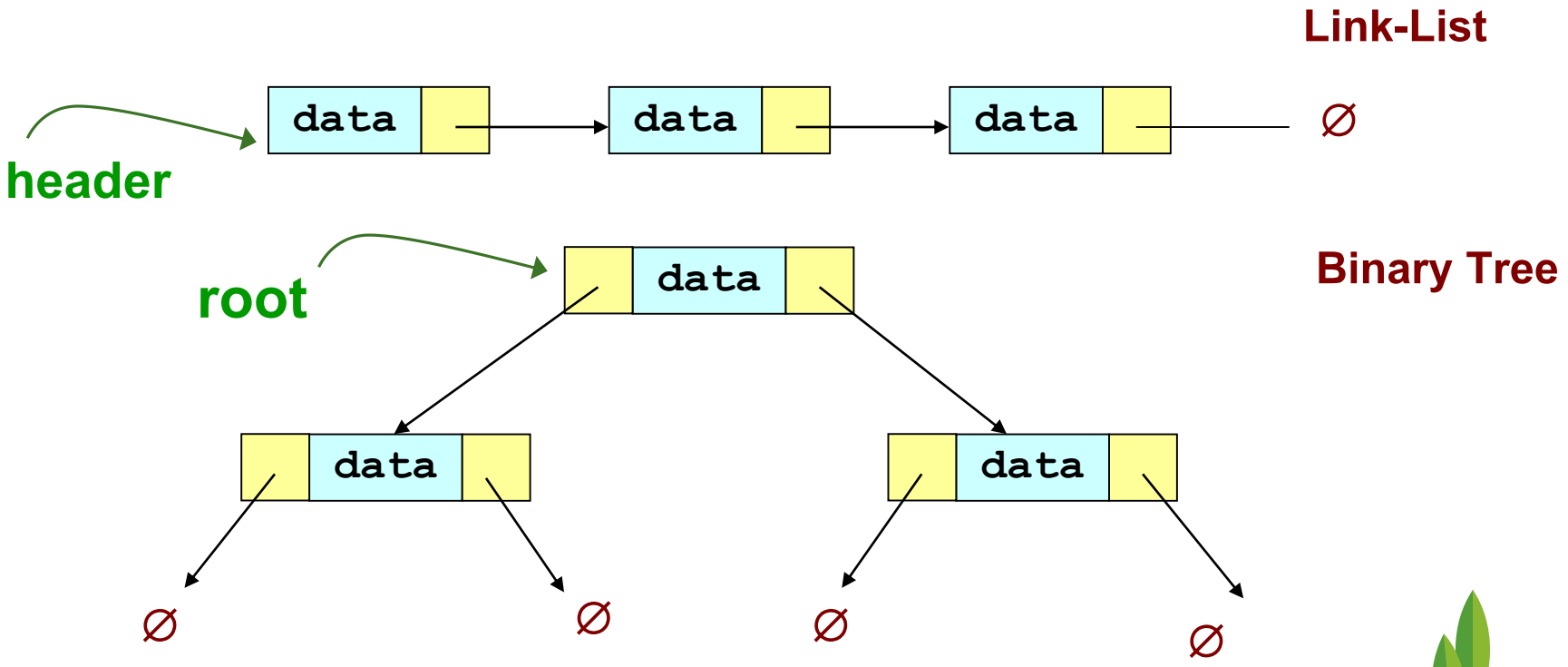

```
breadth-first(q) {
  q.enqueue(p)
  while (!q.isEmpty())
    q.dequeue(p)
    Print item in node p
    for each child w of p
          q.enqueue(w)
}
```


REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Binary Tree Construction : Pointer based

- การสร้าง Binary Tree คล้าย Linked-List แต่เปลี่ยนจาก Next ให้กลายเป็น left subtree กับ right subtree

```
#define itemType int

struct Node{

    itemType data;

    Node*left, *right;

} ;

 Node *root = nullptr, *ptr;
```



REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Example of Binary Tree Construction : Pointer based (using Struct)

```
#include <bits/stdc++.h>
using namespace std;
#define itemType int

struct Node{
    int data;
    Node*left, *right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
} ;
//Global variable
Node *root = nullptr, *ptr;
```

```
Node*createNode (int data) {
    Node *newnode=(Node*)malloc(sizeof(node));
    //ใช้ Node *newnode= new node; ได้
    newnode -> data= data;
    newnode -> left = nullptr;
    newnode -> right = nullptr;
    return(newnode);
}
```

ตัวอย่างการเรียกใช้ฟังก์ชันสร้างโหนดใหม่

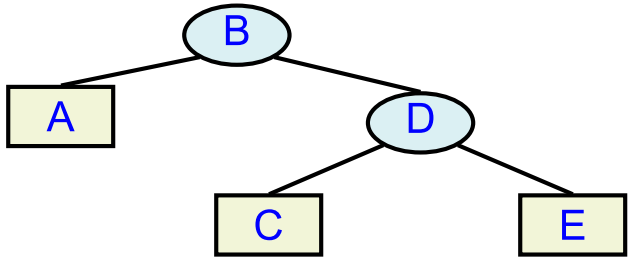
```
ptr = createNode(item);
```

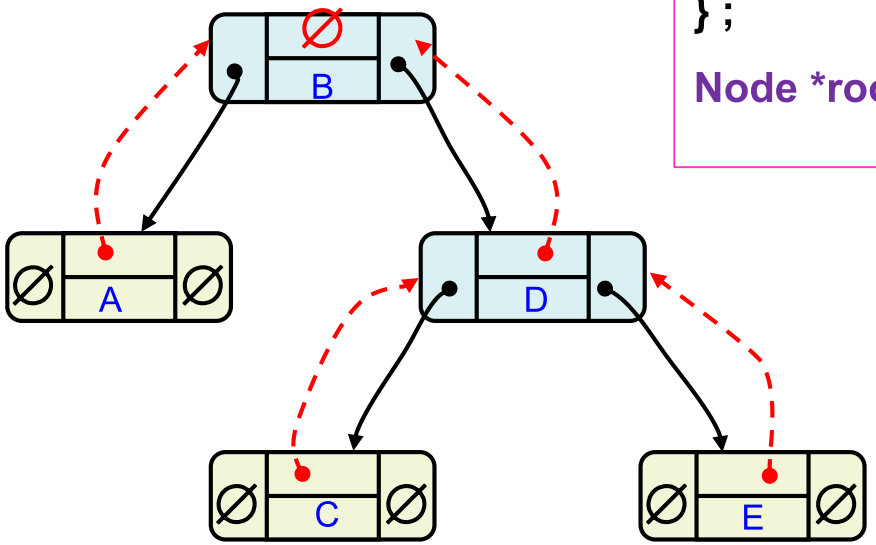

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Implementation for Binary trees : Pointer based

- A node is represented by an object storing
  - ✓ Element
  - ✓ Parent node
  - ✓ Left child node
  - ✓ Right child node

```
#define itemType int
struct node {
        itemType item;
        nodeType *parent;
        Node *left, *right;
} ;
Node *root = nullptr, *ptr;
```

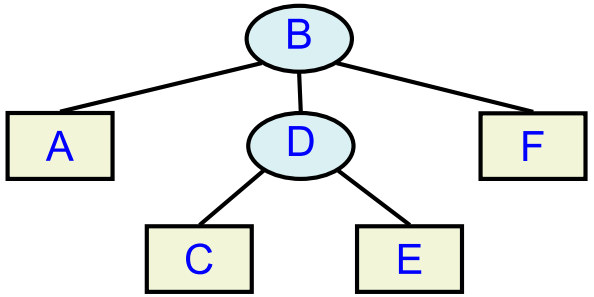


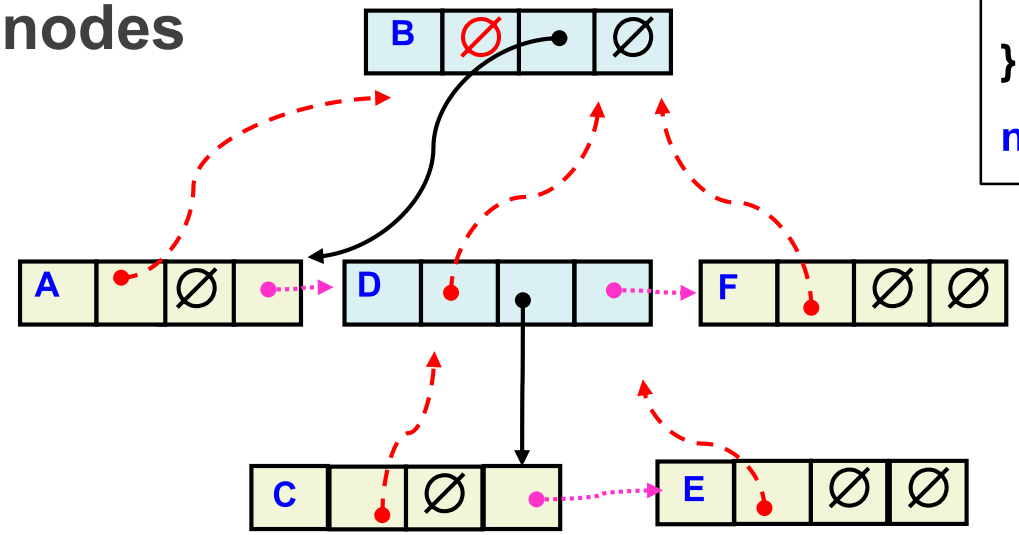


Implementation for trees : Pointer based

- A node is represented by an object storing
  - ✓ Element
  - ✓ Parent node
  - ✓ Sequence of children nodes

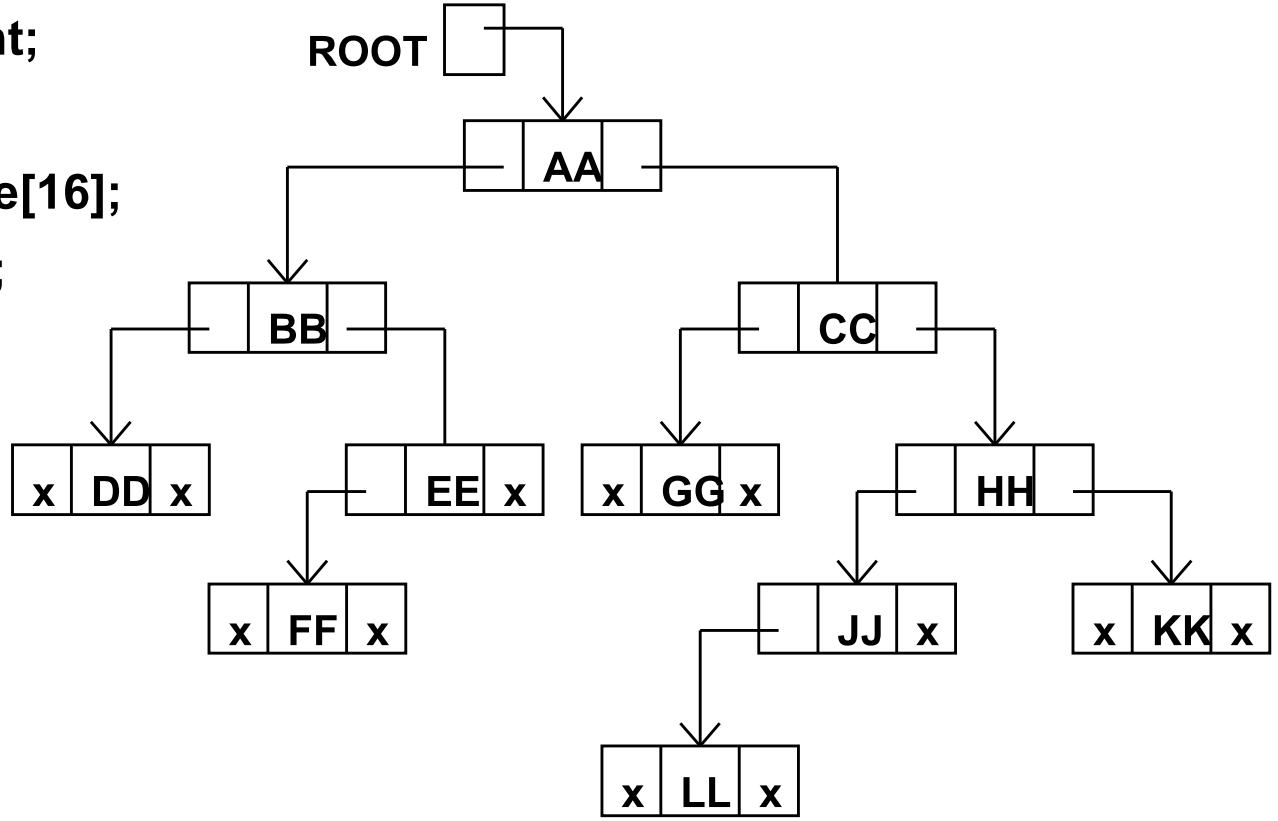
```
#define itemType int
struct node {
    itemType item;
      Node *parent;
      Node *firstchild;
    Node *sibling;
};
node  *root= nullptr, *ptr;
```





Implement Binary tree : Linear Array

```
struct nodeType {
   char item[10];
    int left, right;
} ;
nodeType tree[16];
int root, avail;
```



		item	left	right
	0			
root	1	AA	2	3
	2	BB	4	5
	3	CC	7	8
	4	DD	0	0
	5	EE	6	0
	6	FF	0	0
	7	GG	0	0
	8	HH	9	10
	9	JJ	11	0
	10	KK	0	0
	11	LL	0	0
avail	12		13	0
	13		14	
	14		15	
	15		0	

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

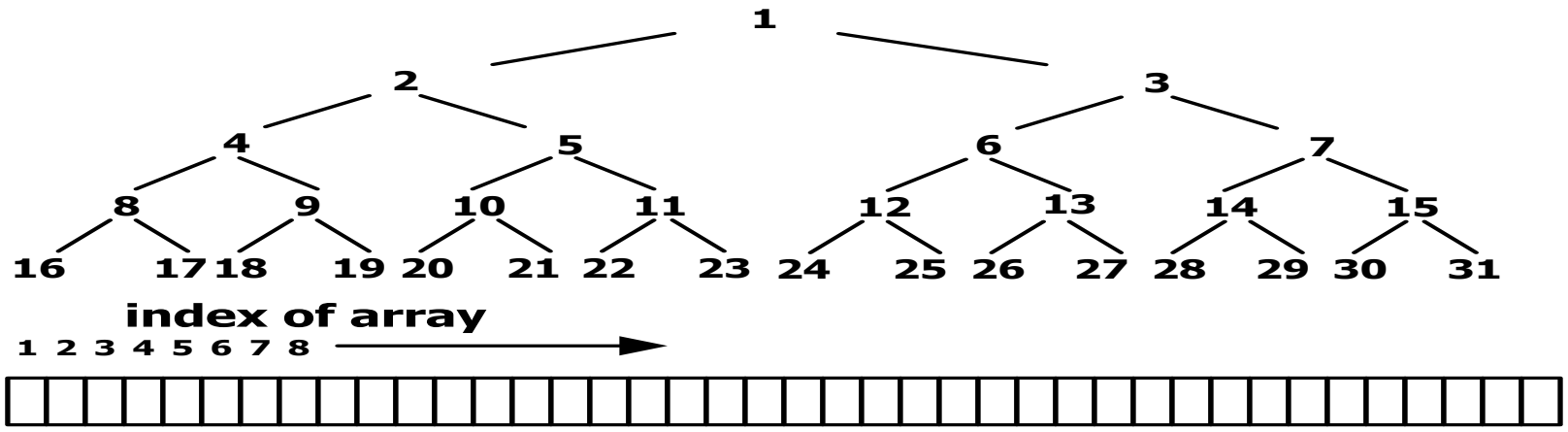
Implement Binary tree : Sequential Array

Address of root node = 1

Address of any node = K

Address of left node = 2*K

Address of right node = 2*K + 1



เช่น K = 5

โหนดทางซ้ายของโหนด 5 คือ 10

โหนดทางขวาของโหนด 5 คือ 11

เริ่ม index 0

Address of root node = 0

Address of any node = K

Address of left node = ?

Address of right node = ?

Implement Binary tree : Sequential Array

Address of root node = 1

Address of any node = K

Address of left node = 2*K

Address of right node = 2*K + 1

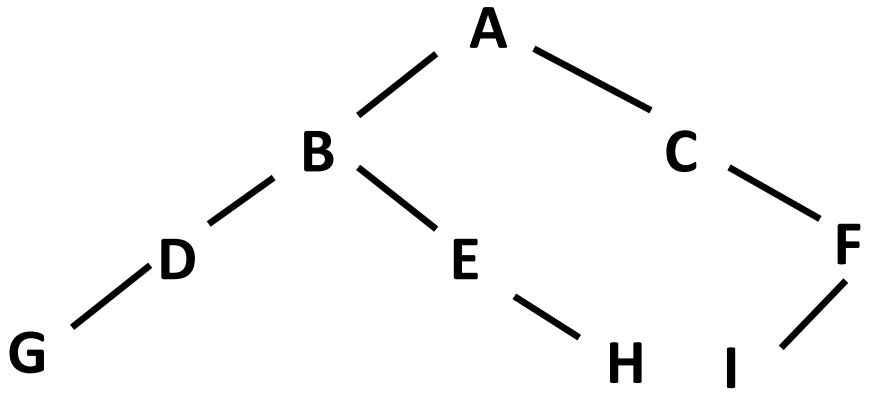
Address of node E = 2*Address of node B + 1

Address of node B = 2*Address of Root A

Address of root node A = 1

Address of node B = 2*1 =2

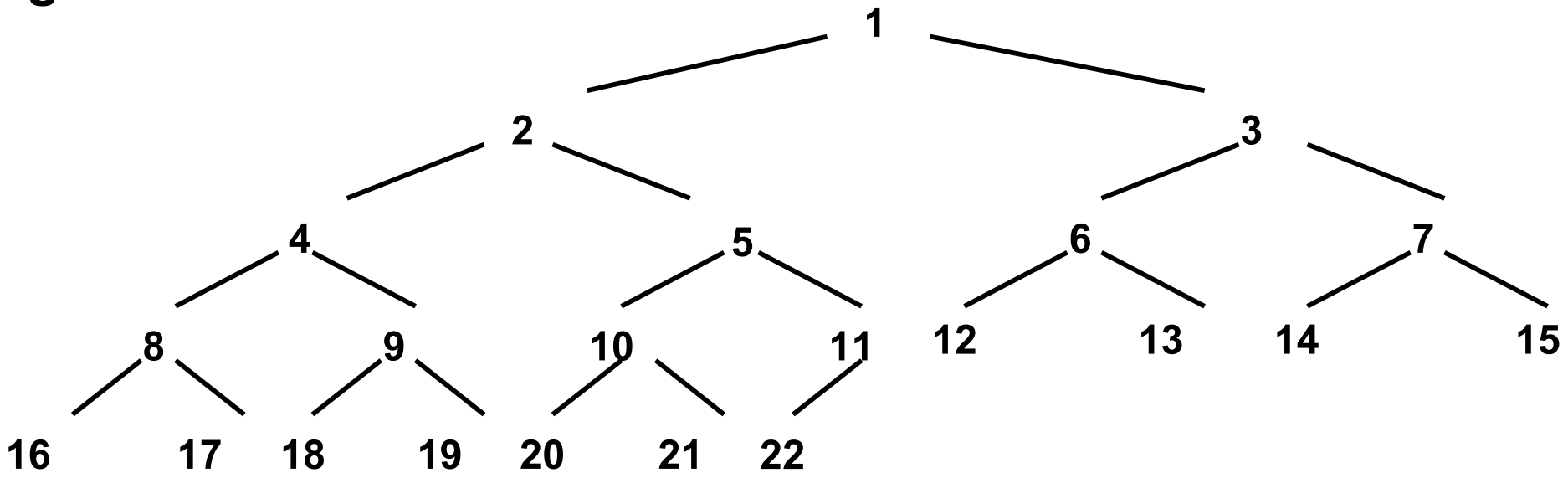
Address of node E = 2*2 + 1 = 5



A	B	C	D	E		F	G			H			I
1	2	3	4	5	6	7	8	9	10	11	12	13	14

Complete Binary Tree

- A binary tree which the nodes are labeled in inorder sequence from the left to right.



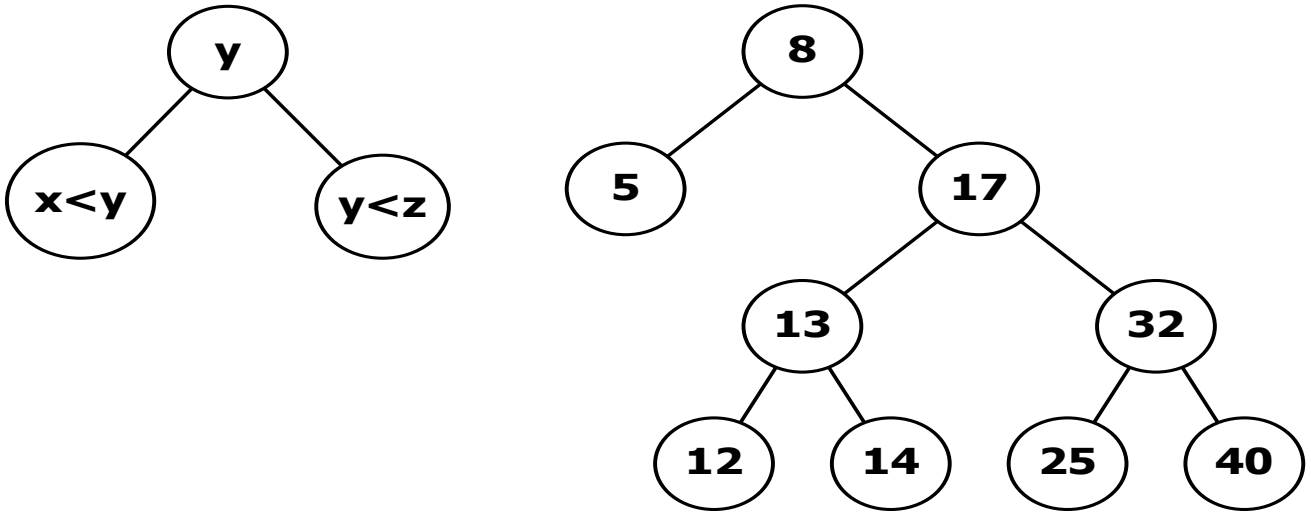
REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Binary Search Tree

Binary Search Tree

➢ **Binary SearchTree มีคุณสมบัติดังนี้**

"all keys in the left subtree < root < all keys in the right subtree"



จำนวน node สูงสุดของ tree ที่มีความสูง k จะมีค่าเท่ากับ

$2^{(k+1)}$-1 เช่น depth = 2 จำนวน node สูงสุด = $2^{(2+1)}$-1 = 7

Binary Search Tree : Operation

Insert node

- ถ้า key ของโหนดใหม่ > key ของ current node x
  - ✓ให้ move ไปยัง right child ของโหนด x
- ถ้าไม่ใช่ก็ให้ move ไปยัง left child ของโหนด x
- ทำซ้ำจนกระทั่งพบว่าโหนด x เป็นค่า nullptr

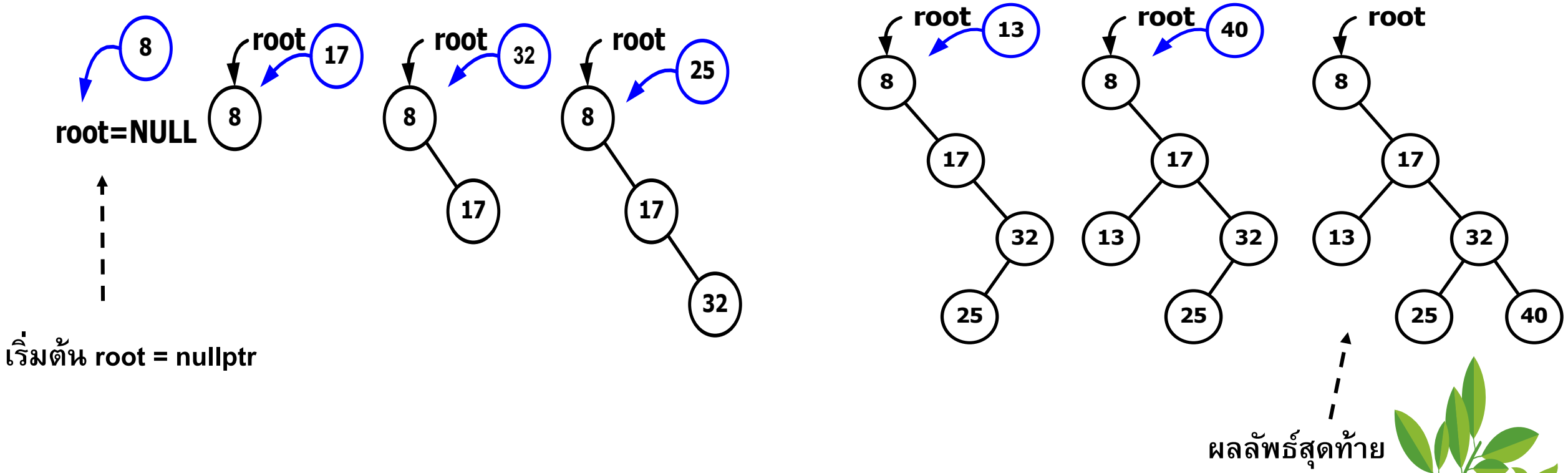
  ➔ พบตำแหน่งที่ต้องการแทรกโหนดใหม่
- ให้ทำการเพิ่มโหนดใหม่เป็น left child หรือ right child node ของโหนดที่เป็น *x's* parent
  - ✓ ตำแหน่ง left หรือ right ขึ้นอยู่กับค่า key ของโหนดที่เป็น *x's* parent

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Binary Search Tree : Operation

Insert node

- แสดงลำดับของการแทรกโหนด 8 17 32 25 13 40



REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

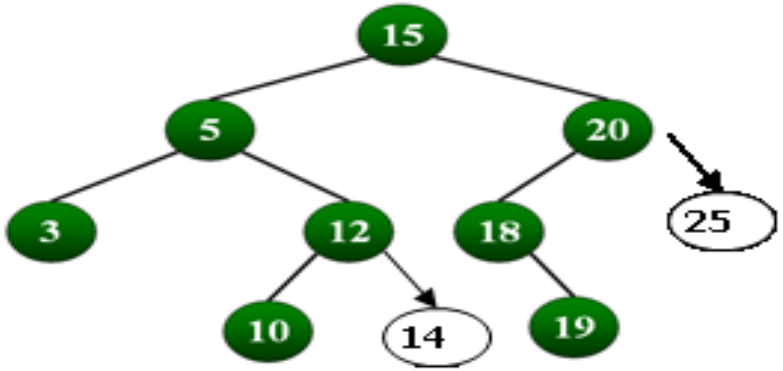
Binary Search Tree : Operation

แบบฝึกหัด 1

- ในการสร้าง Binary Search Tree จากข้อมูลต่อไปนี้

  15, 5, 12, 20, 3, 18, 10, 19, 14, และ 25

  โครงสร้างของ binary tree ที่ได้คือ ?



- หาผลลัพธ์ของ Tree traversal ในรูปแบบ ต่าง ๆ ?

- Preorder (Root -> ซ้าย -> ขวา) ผลลัพธ์คือ 15 5 3 12 10 14 20 18 19 25
- Inorder (ซ้าย -> Root -> ขวา) ผลลัพธ์คือ 3 5 10 12 14 15 18 19 20 25
- Postorder (ซ้าย -> ขวา -> Root) ผลลัพธ์คือ 3 10 14 12 5 19 18 25 20 15

Binary Search Tree Implementation : Link List


```
#include <bits/stdc++.h>
using namespace std;
struct Node{
    int data;
    Node* left, *right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
} ;
```

ตัวอย่างการเรียกใช้ insert()

```
Node *root = nullptr;

กรณีเราประกาศ global
if (root==nullptr){
  //กรณีโหนดแรก
  root=new Node (input);
}
else{
  insert(input);
}
```

สามารถปรับการเรียกใช้เป็น ส่งเป็น parameter

```
root = insert(root, input);
```

Binary Search Tree Implementation : Link List

Insert a new node

```
void insert(Node *root, int data) {
        Node *p, *previous ;
        p = root ;

        while (p!=nullptr) {



        }



}
```

หาโหนดสุดท้ายเก็บไว้ใน previous

เลือกตัวชี้ที่จะต้อง link ไปยังโหนดใหม่

Binary Search Tree Implementation : Link List

Insert a new node

การเรียกใช้เมท็อด insert()

```
cin >> input;
if(root==nullptr){
   root=createNode(input); //กรณีโหนดแรก
}
else{
   insert(root, createNode(input));
}
```

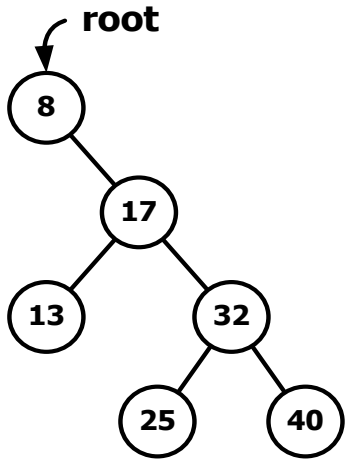
ตัวอย่างเมท็อด insert() แบบ recursive

และส่งโหนดข้อมูลแทน ตัวแปร data

```
void insert(Node *p, Node *newNode){
    if(newNode ->data < p->data){
        if(p->left==nullptr)
            p->left = newNode;
        else
            insert(p->left, newNode);
    }else {
        if(p->right==nullptr)
            p->right= newNode;
        else
            insert(p->right, newNode);

    }
}
```

Binary Search Tree : Basic Operation

ทำต่อใน ข้อ C2P23 : Program



จงเขียนโปรแกรมในการรับเลขจำนวนเต็ม N จำนวนเพื่อสร้าง Binary Search Tree

6

8 17 32 25 13 40

แล้วแสดงค่าของทุกโหนดในทรีที่การเข้าถึงทุกโหนดในทรี (Traversal) แบบต่าง ๆ ต่อไปนี้

- preOrder(root) — 8 17 13 32 25 40
- inOrder(root) — 8 13 17 25 32 40
- postOrder(root) — 13 25 40 32 17 8
- pathMin() — 8
- pathMax() — 8 17 32 40
- Leaves() — 13 25 40

Binary Search Tree : Operation

Delete node

แยกเป็น 3 กรณี

1. x เป็นโหนด leaf — ให้โหนดที่เป็น parent ของ x (*x*'s parent) ชี้ไปยังค่า nullptr
2. x มีโหนดลูกเพียงโหนดเดียว — ให้โหนดที่เป็น parent ของ x ชี้ไปยังโหนดลูกของ x (*x*'s child)
3. x มี two children — หาตำแหน่งโหนด *y* ที่เป็น leftmost node ของ *x*'s right subtree (key ของโหนด y จะเป็น inorder successor ของ key ของโหนด x)
   - ✓ ให้ copy ค่า item จากโหนด y ไปเก็บที่โหนด *x*
   - ✓ จากนั้นให้ลบโหนด y โดยแยกเป็น 2 กรณีคือ

   3.1 ถ้า y เป็นโหนด leaf ให้โหนดที่เป็น parent ของ x (*x*'s parent) ชี้ไปยังค่า nullptr

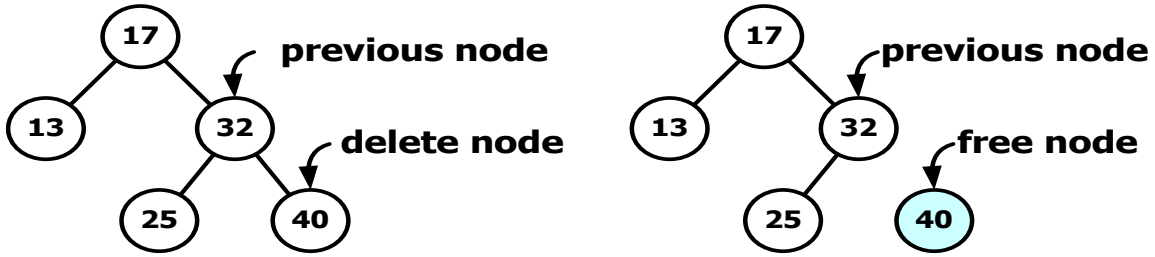
   3.2 ถ้า y เป็น internal node (has one child)

   - Let *y*'s parent adopt *y*'s child

Binary Search Tree : Operation

Delete node: Case 1

- การลบ leaf node




```
x = previous -> right;

previous ->right = nullptr;

free(x);
```

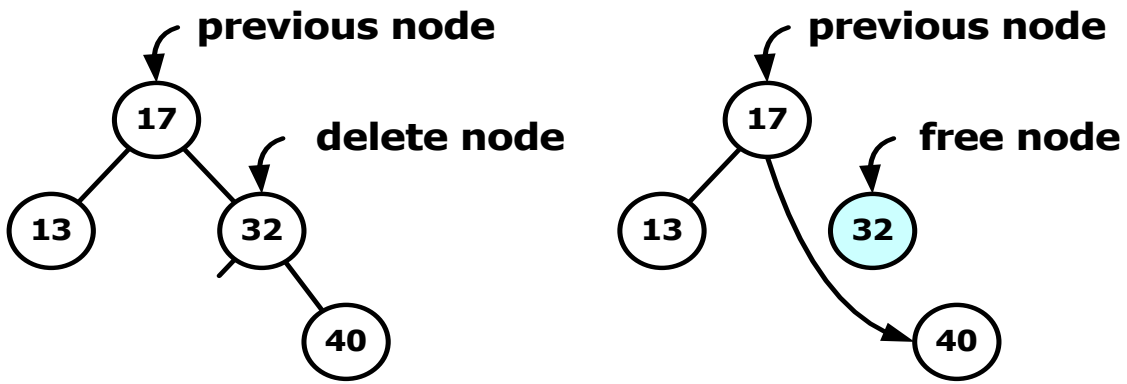
previous ชี้ที่โหนด 32
ดังนั้น x คือ โหนด 40

ลบทางขวาของโหนด 32

Binary Search Tree : Operation

Delete node: Case 2

- การลบโหนดที่มีเพียง 1 child




```
x = previous -> right;

previous ->right = x->right;

free(x);
```

previous ชี้ที่โหนด 17
ดังนั้น x คือ โหนด 32

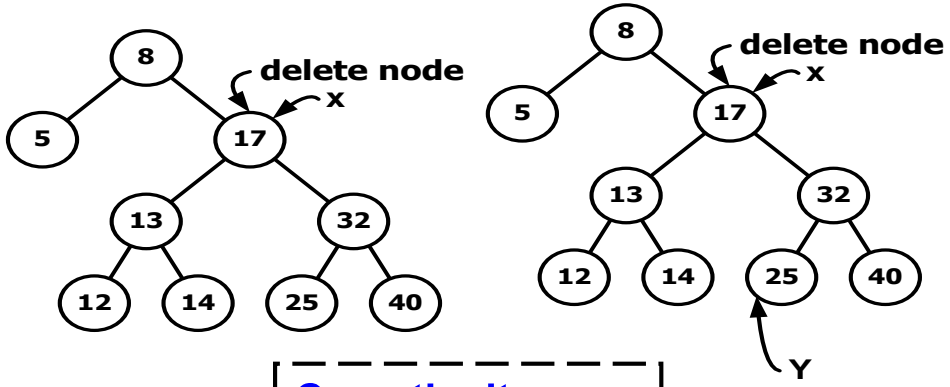
กำหนดให้ทางขวาของ 17
ชี้ไปทางขวาของโหนด 32 (40)

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Binary Search Tree : Operation

Delete node: Case 3.1

- การลบโหนดที่มี child มากกว่า 1 (non-terminal node)

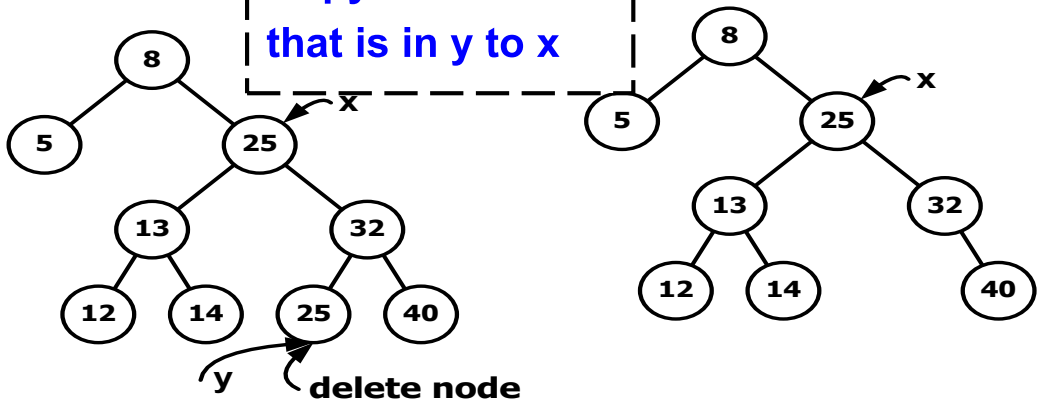


หา leftmost node ของ x's right subtree

(y =โหนด 25)

Copy the item

that is in y to x



Remove node y by

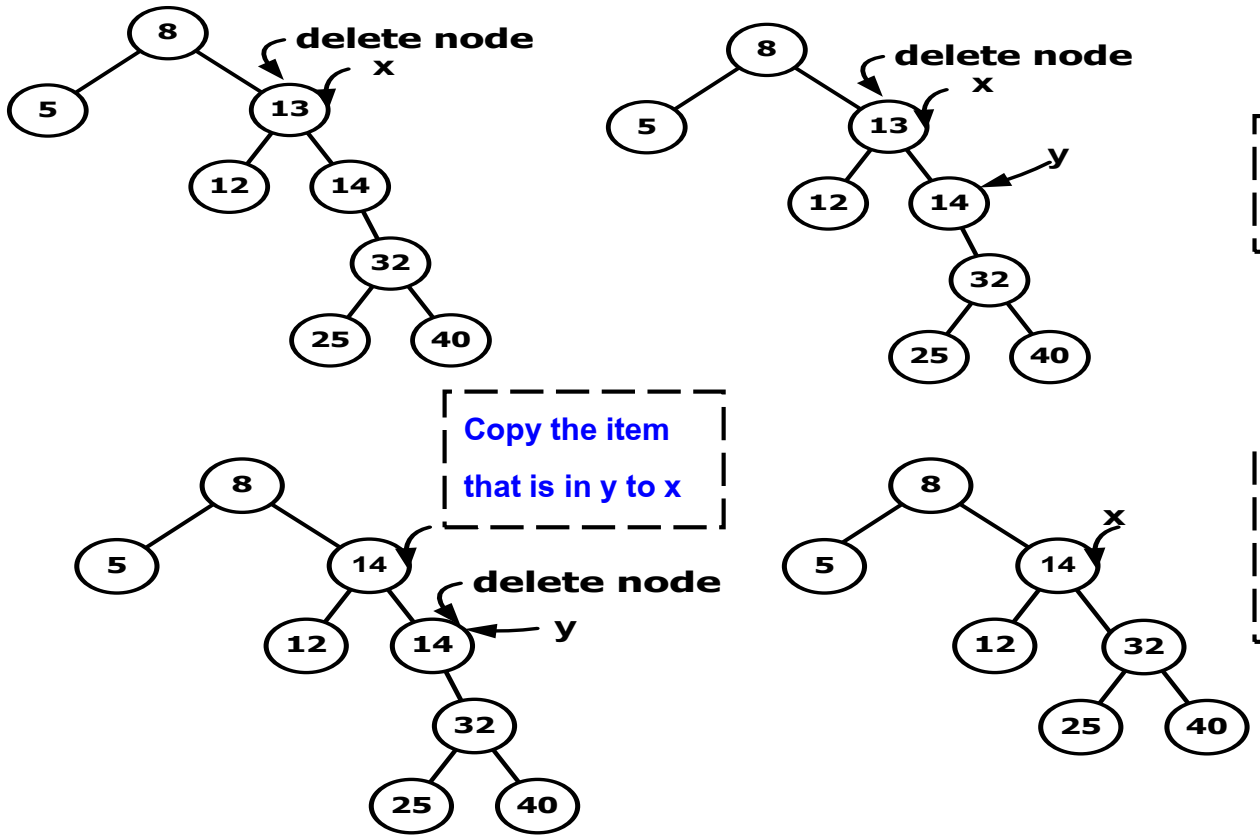
3.1) y is a leaf

Set the pointer in y's parent to nullptr

Binary Search Tree : Operation

Delete node: Case 3.2

- การลบโหนดที่มี child มากกว่า 1 (non-terminal node)



หา leftmost node ของ x's right subtree
(y =โหนด 14)

Remove node y by
3.2) y has one child
Let y's parent adopt y's child

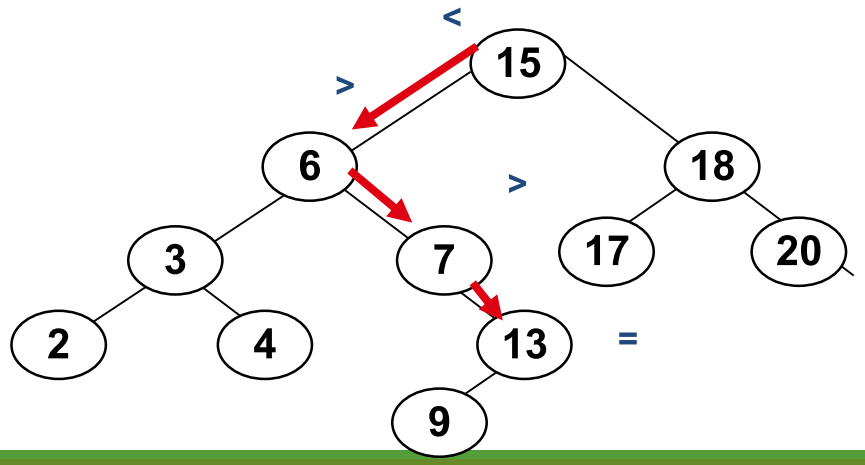
REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Binary Search Tree : Operation

Find key

- เริ่มต้นที่โหนด root
- เปรียบเทียบค่า key ที่ค้นหากับค่า key ของโหนด
  - ✓ หากพบค่าที่ค้นหาให้คืนค่าโหนด
  - ✓ หากยังไม่พบให้ค้นหาไปยังโหนดลูกต่อไป
  - ✓ หากเจอโหนดที่เป็น leaf แล้วยังไม่พบ ก็จะ return ค่า nullptr
- เช่น Find(root,13)

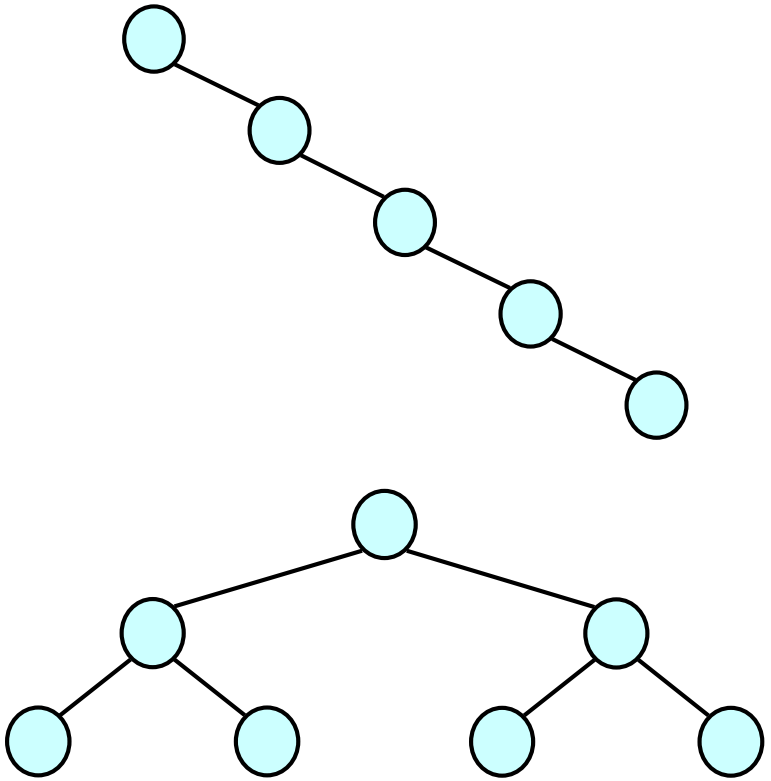
```
Algorithm find ( v, 13)
    if  isnullptr(v)
        return nullptr
    if k < key(v)
        return find(k,left (v))
    else if k = key(v)
        return v
    else { k > key(v) }
        return find(k,right (v))
```



Binary Search Tree Performance

For a tree with n nodes

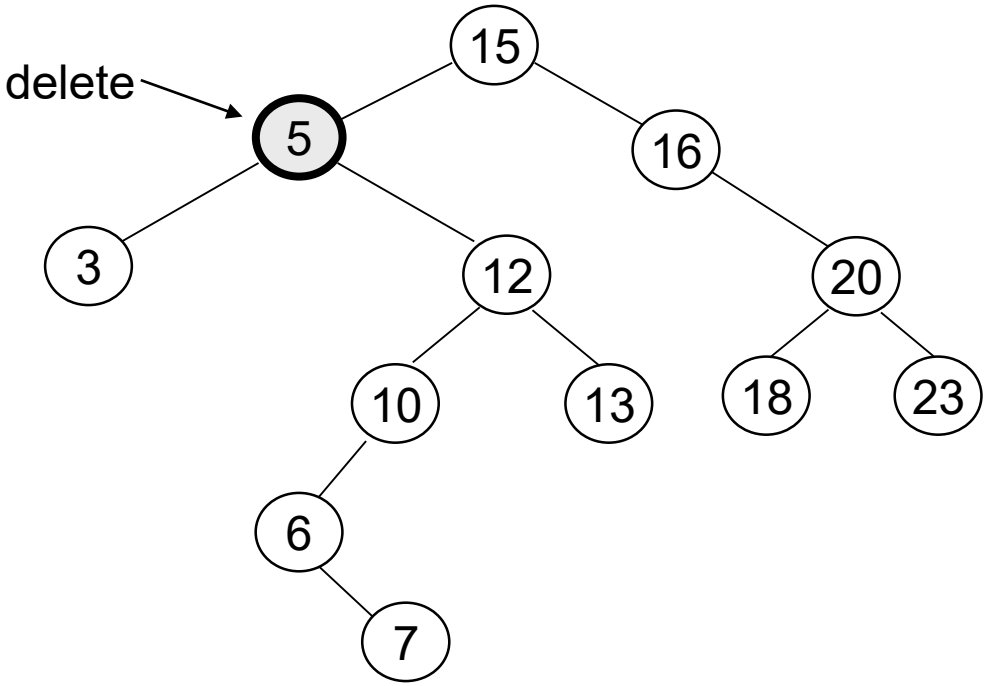
- The space used is $O(n)$
- For methods : find, insert and remove
  - ✓ The time taken is $O(h)$
  - ✓ Worst case ➔ ความสูง (h) เท่ากับ n
    - $O(n)$
  - ✓ Best case
    - $O(\log n)$



Binary Search Tree : Operation

แบบฝึกหัด 2

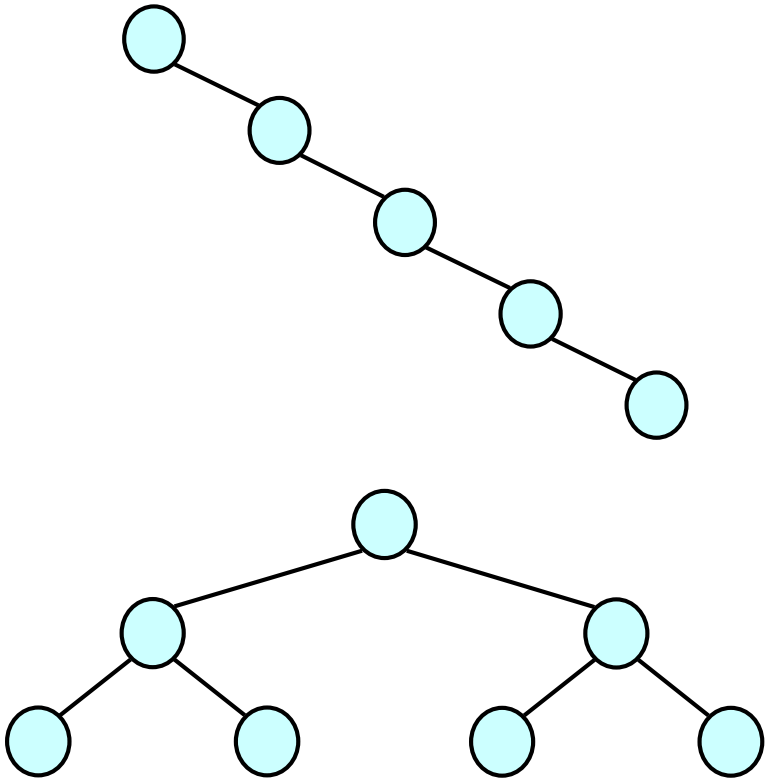
- Leaf nodes คือ ?
- Root ของ right subtree โหนด 5 คือ ?
- Successor ของโหนด 5 คือ ?
- Successor ของโหนด 15 คือ ?
- หลังลบ 5 โครงสร้างทรีคือ ?



Binary Search Tree Performance

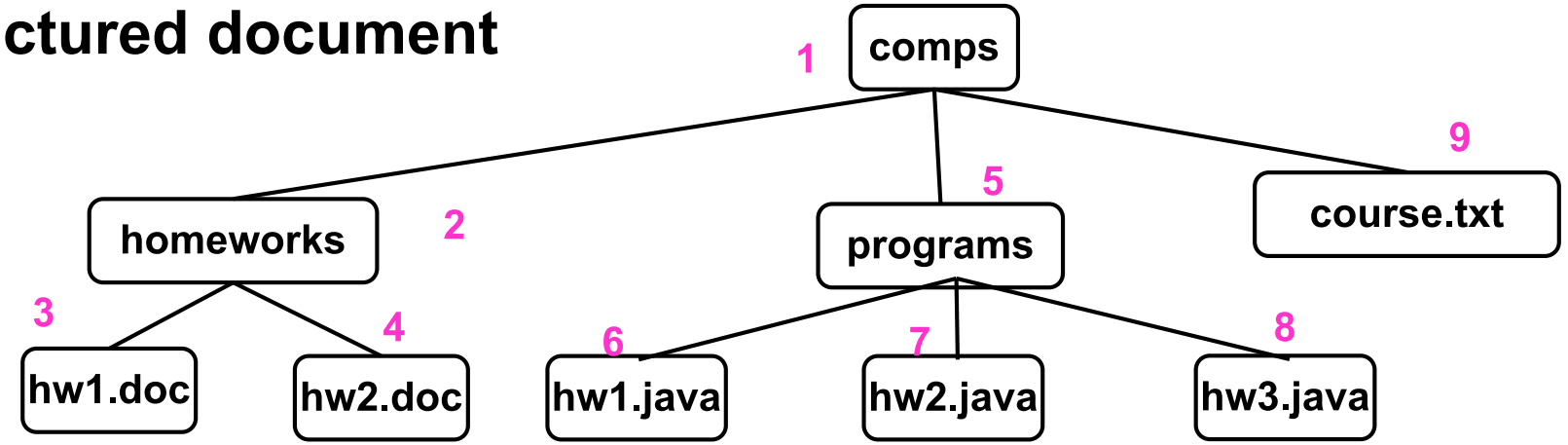
For a tree with n nodes

- The space used is $O(n)$
- For methods : find, insert and remove
  - ✓ The time taken is $O(h)$
  - ✓ Worst case ➔ ความสูง (h) เท่ากับ n
    - $O(n)$
  - ✓ Best case
    - $O(\log n)$



Tree Application : File systems

- Print a structured document



preOrder →

```
Comps
  homeworks
    hw1.doc
    hw2.doc
  programs
    hw1.java
    hw2.java
    hw3.java
  course.txt
```

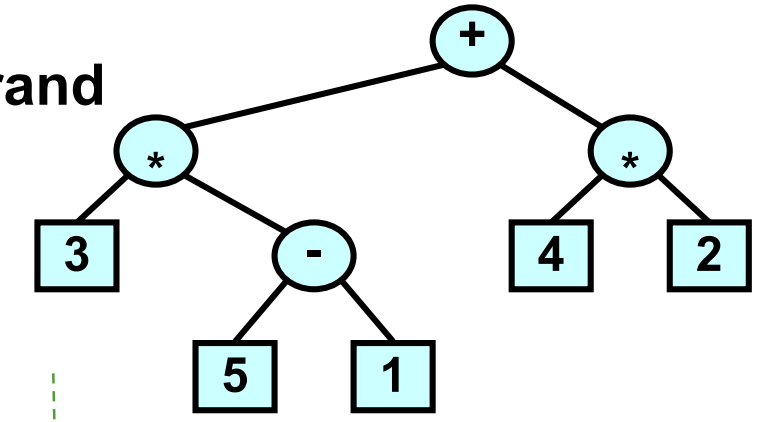
postOrder →

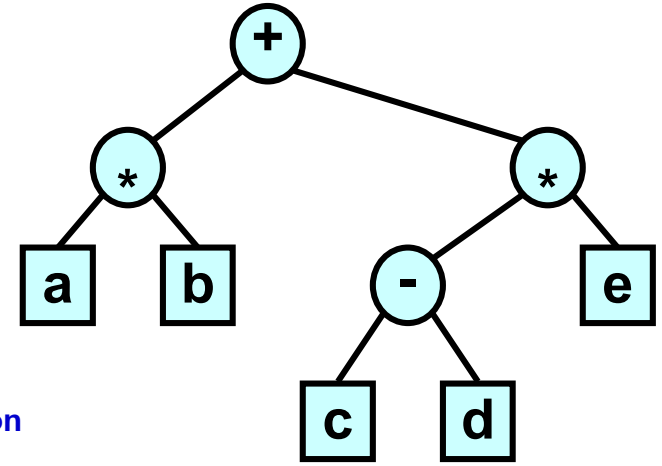
```
    hw1.doc
    hw2.doc
  homeworks
    hw1.java
    hw2.java
    hw3.java
  programs
  course.txt
Comps
```


REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Applications of Binary Tree

- Binary tree associated with an arithmetic expression
  - ✓ Internal node : Operator
  - ✓ โหนดใบ (External node) : Operand
- Subtree : subexpression





- การเข้าถึงทุกโหนด (Traversal)
  - ✓ preOrder ..... + * 3 – 5 1 * 4 2 ..... +*ab*-cde ..... Prefix Expression
  - ✓ postOrder ..... 3 5 1 - * 2 y * + ... ab*cd-e*+ ..... Postfix Expression
  - ✓ inOrder ..... 3 * 5 - 1 + 4 * 2 ..... a*b+c-d*e ..... Infix Expression

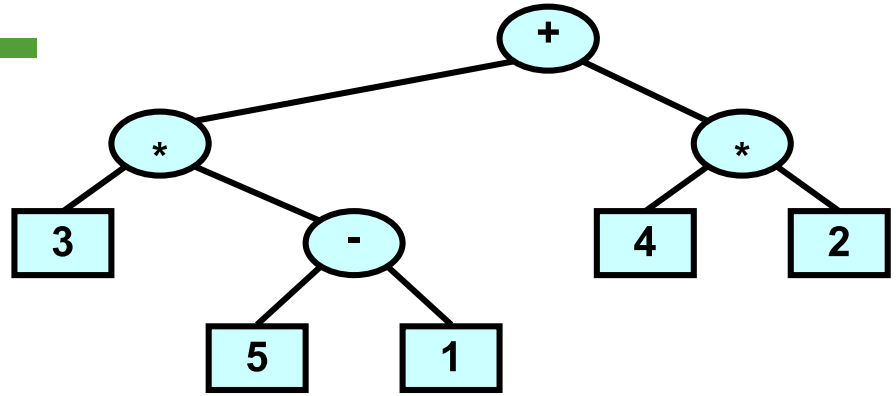
REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

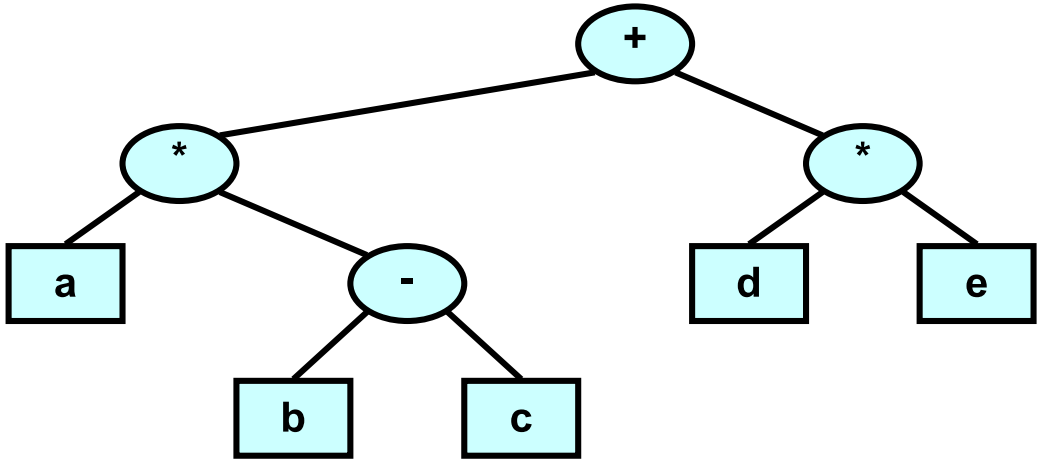
Applications of Binary Tree

inOrder Traversal

- 3 * 5 - 1 + 4 * 2
- a * b – c + d * e

```
inOrder(p) {
  if hasLeft (p)
      inOrder (leftchild (p))
  Print item in node p
  if hasRight (p)
      inOrder (rightchild (p))
}
```





- ✓ (3 * (5 - 1) + (4 * 2))
- ✓ (a * (b - c))+ (d * e))

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

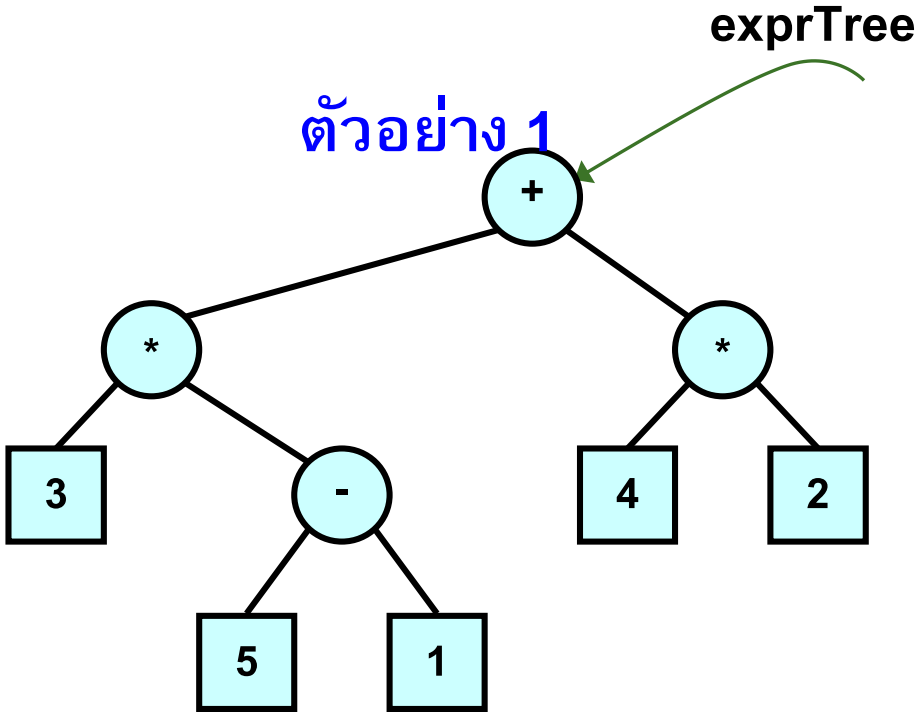
Applications of Binary Tree

จงเขียนเมท็อด printExpr() เพื่อพิมพ์ Expression จาก Arithmetic Expression Tree

✓ ((a * (b + c))+ d)

Algorithm for printing an arithmetic expression

```
printExpr(input p : BinaryTree){

}
```

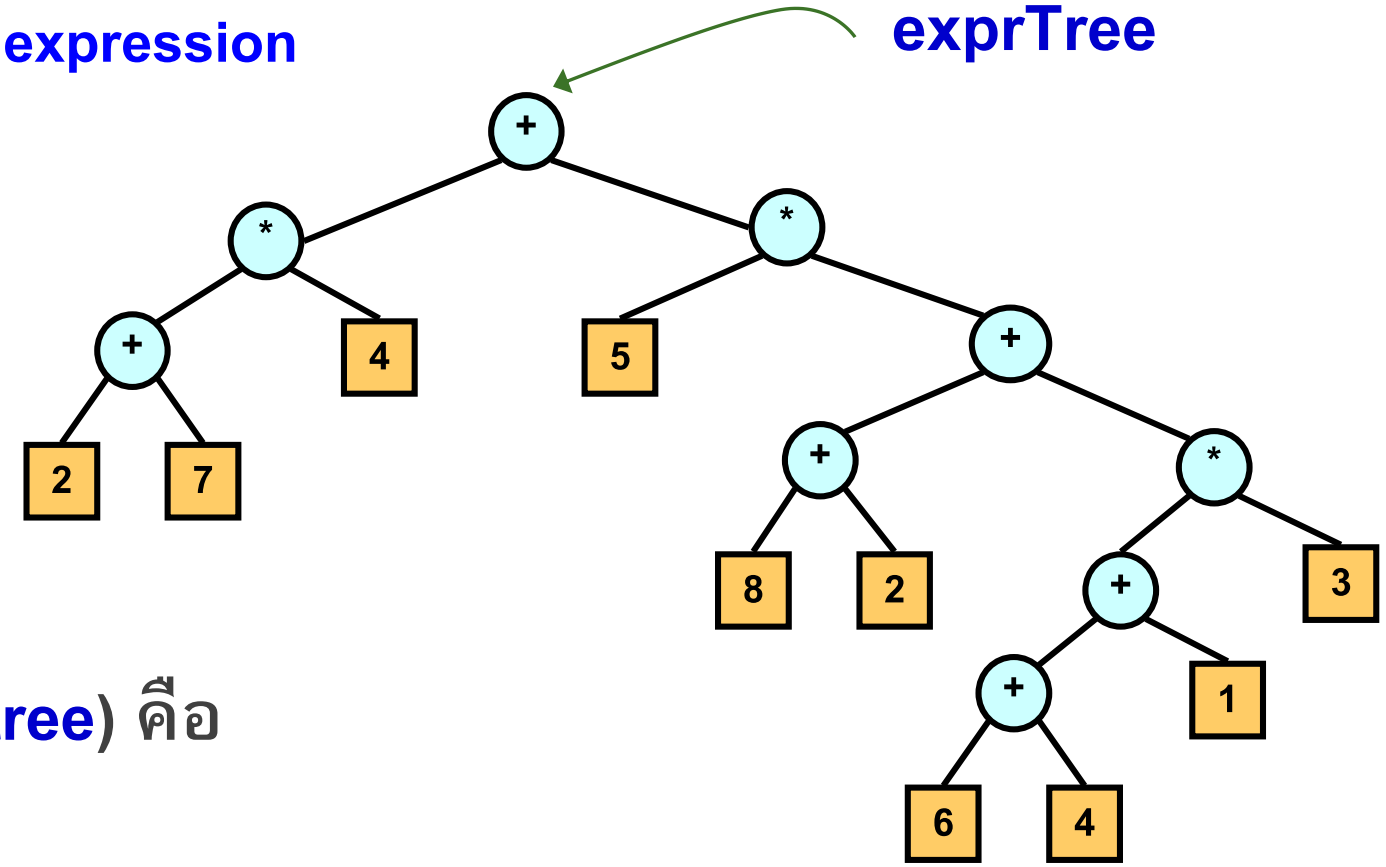


((3 * (5 - 1)) + (4 * 2))

ตัวอย่างผลลัพธ์ printExpr(exprTree)

Applications of Binary Tree: แบบฝึกหัด 2

ตัวอย่าง 2 การแสดงประโยค expression



➢ ผลลัพธ์ printExpr(exprtree) คือ

(((2 + 7) * 4) + (5 * ((8 + 2) + (((6 + 4) + 1) * 3))))

Applications of Binary Tree : LAB3

แปลงค่าสตริง Postfix Expression เป็น Expression tree

- ดังตัวอย่าง

Hint: เก็บเป็นอาร์เรย์ได้

vector <Node*> expArr;

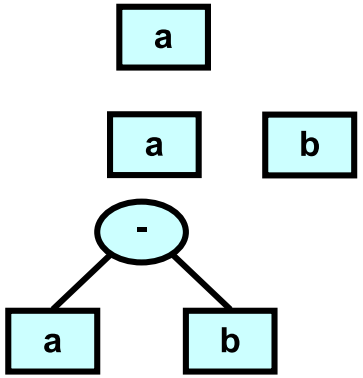
➢ ให้แสดงลำดับการสร้าง Expression tree ของPostfix Expression ต่อไปนี้

✓ ABC*+D+

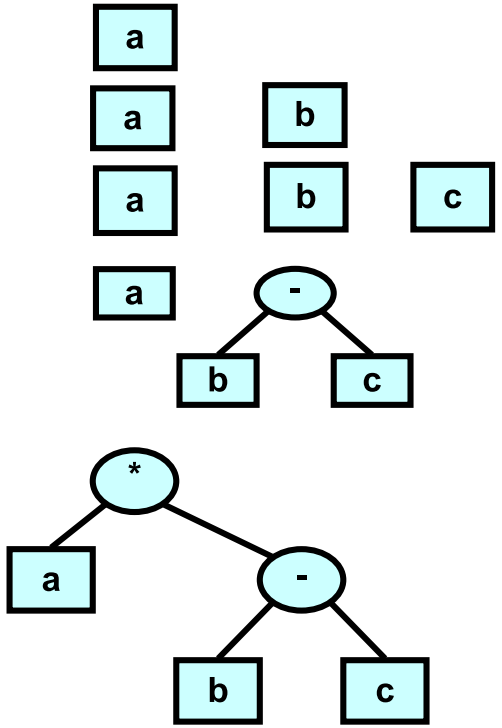
✓ AB+CD+*

-ถ้ารับ input เป็น ab*

-จะสร้างทรี 3 ขั้นตอน ดังนี้



- ถ้ารับ input เป็น abc-*
- จะสร้างทรีด้วย 5 ขั้นตอน ดังนี้

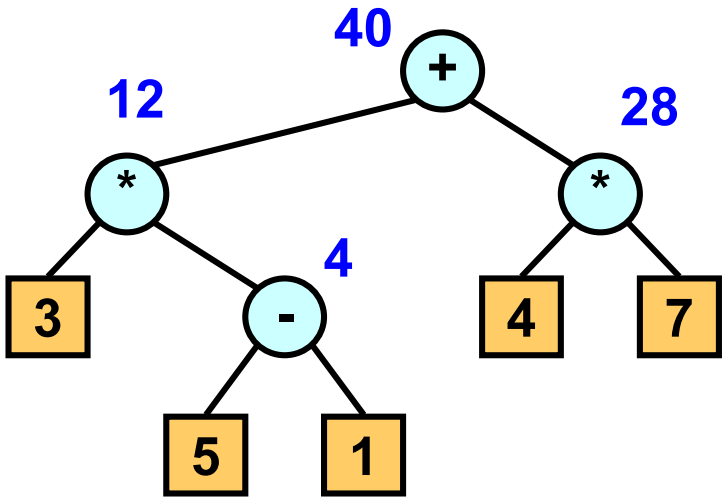


Applications of Binary Tree : LAB3

Algorithm for evaluating an arithmetic expression

```
ItemType eitemExpr(BinaryTree p)
{   var a, b : ItemType
    if isExternal(p)
        return p.getItem()
    else
        a ← eitemExpr(leftChild(p))
        b ← eitemExpr(rightChild(p))
        //operator stored at node p
        Δ ← p.getItem()
        return Compute(a, Δ, b)
}
```

Arithmetic Expression Tree



((3 * (5 - 1)) + (4 * 7)) ➔ 40

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Applications of Binary Tree : Huffman coding

A Character-coding

การเข้ารหัส (Encoding) ข้อความ

- การเปลี่ยนข้อความ message ของเซตตัวอักษร เป็นกลุ่มเลขฐานสอง โดยแทนค่าแต่ละตัวอักษร ด้วยเซตของกลุ่มบิตที่เรียกว่า codewords

ตัวอย่างเช่น เซตของตัวอักษร = {a, b, c, d}

หากเข้ารหัสด้วยเซต C { a=00, b = 01, c=10, d =11}

ข้อความ “cab” จะถูกเข้ารหัสเป็น 100001

หากเข้ารหัสเซต C { a=0, b = 110, c=10, d =111}

ข้อความ “cab” จะถูกเข้ารหัสเป็น 100110

การถอดรหัส (decoding) ข้อความ

- ✓ การเปลี่ยนค่าจาก ข้อความที่เข้ารหัสอยู่ (encoded message) กลับไปเป็นข้อความเดิม (original message)

Applications of Binary Tree : Huffman coding

A Character-coding Problem1

- การเข้ารหัสข้อมูลแบบความยาวคงที่ (Fixed-length)
  - ✓ ข้อมูลหนึ่งตัวอักษรใช้เนื้อที่ (จำนวนบิตของรหัสที่แทนข้อมูล) เท่ากันหมด เช่น รหัส ASCII
- การเข้ารหัสข้อมูลแบบความยาวแปรได้ (Variable-length)
  - ✓ จำนวนบิตของรหัสที่ใช้แทนแต่ละข้อมูลอาจไม่เท่ากันได้ เช่น รหัสฮัฟฟ์แมน
- สมมุติว่าแฟ้มข้อมูลหนึ่งมี 100,000 อักขระ ประกอบด้วยตัวอักษร a b c d e และ f และมีความถี่ในการปรากฏของแต่ละตัวอักษรในแฟ้มข้อมูลเป็น 45, 13, 12, 16, 9, และ 5 เปอร์เซ็นต์ ตามลำดับ

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Applications of Binary Tree : Huffman coding

A Character-coding Problem1

- หากมีการเข้ารหัสแบบความยาวคงที่ และแบบแปรได้ ดังนี้

	a	b	c	d	e	f
Frequency	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

พบว่า Fixed-length ด้วย 3-bit codeword ใช้เนื้อที่รวม 300,000 บิต

ส่วนแบบ Variable-length ใช้เนื้อที่รวม

(45*1+13*3+12*3+16*3+9*4+5*4)/100*100,000 = 224, 000 บิต

ลดลง ≈25 %

Applications of Binary Tree : Huffman coding

A Character-coding Problem1

- จุดเด่นของรหัสแบบแปรได้ คือ สามารถกำหนดรหัสสั้น ๆ ให้กับข้อมูลที่ปรากฏบ่อย ๆ จะทำให้ปริมาณหน่วยความจำในการเก็บข้อมูลโดยรวมที่น้อยกว่าการใช้รหัสแบบความยาวคงที่
- ปัญหา คือ หากเรารู้ความถี่ของการปรากฏของข้อมูลของกลุ่มอักขระกลุ่มหนึ่ง (message) แล้ว เราจะหารหัสให้กับชุดข้อมูลที่จะใช้หน่วยความจำ (จำนวนบิต) น้อยที่สุดได้อย่างไร

Applications of Binary Tree : Huffman coding

A Character-coding Problem2

ตัวอย่างในการเข้ารหัสสำหรับข้อความ **bad**

หากกำหนดกลุ่มบิตสำหรับการเข้ารหัส 3 แบบดังนี้

แบบที่ 1 : C_1 = {a = 00, b = 01, c = 10, d = 11}

ข้อความเข้ารหัสที่ได้คือ 010011

แบบที่ 2 : C_2 = { a = 0, b = 110; c = 10, d = 111}

ข้อความเข้ารหัสที่ได้คือ 1100111

แบบที่ 3 : C_3 = { a = 1, b = 110; c = 10, d = 111}

ข้อความเข้ารหัสที่ได้คือ 1101111

ตัวอย่างในการถอดรหัส

จาก C_1 : 010011 สามารถถอดรหัสได้ข้อความ bad แบบเดียว

จาก C_2 : 1100111 สามารถถอดรหัสได้ข้อความ bad แบบเดียว

จาก C_3 : 1101111 สามารถถอดรหัสได้ข้อความมากกว่า 1 แบบ คือ

bad (1101111) หรือ bda (1101111)

หรือ acad (1101111)

Applications of Binary Tree : Huffman coding

A Character-coding Problem 2

- การเข้ารหัสแบบ Fixed-length จะได้ข้อความถอดรหัสที่ unique เสมอ
- แต่เราสนใจการเข้ารหัสแบบ Variable-length มากกว่า

- ปัญหาคือเราจะหาวิธีที่จะเข้ารหัสอย่างไรที่จะทำให้การถอดรหัสของ encoded message จะได้ message เดียว

- คำตอบคือเราจะใช้ codeword แบบ prefix code

Applications of Binary Tree : Huffman coding

Prefix-Codes

- รูปแบบรหัสที่จะไม่มีรหัสใดเป็นส่วนเติมหน้าของรหัสอื่น

- ตัวอย่างรหัสที่ไม่ใช่ prefix code

  { a = 1, b = 110; c = 10, d = 111}

- ตัวอย่างรหัสแบบ prefix code

  {a = 0, b = 110, c =10, d =111}

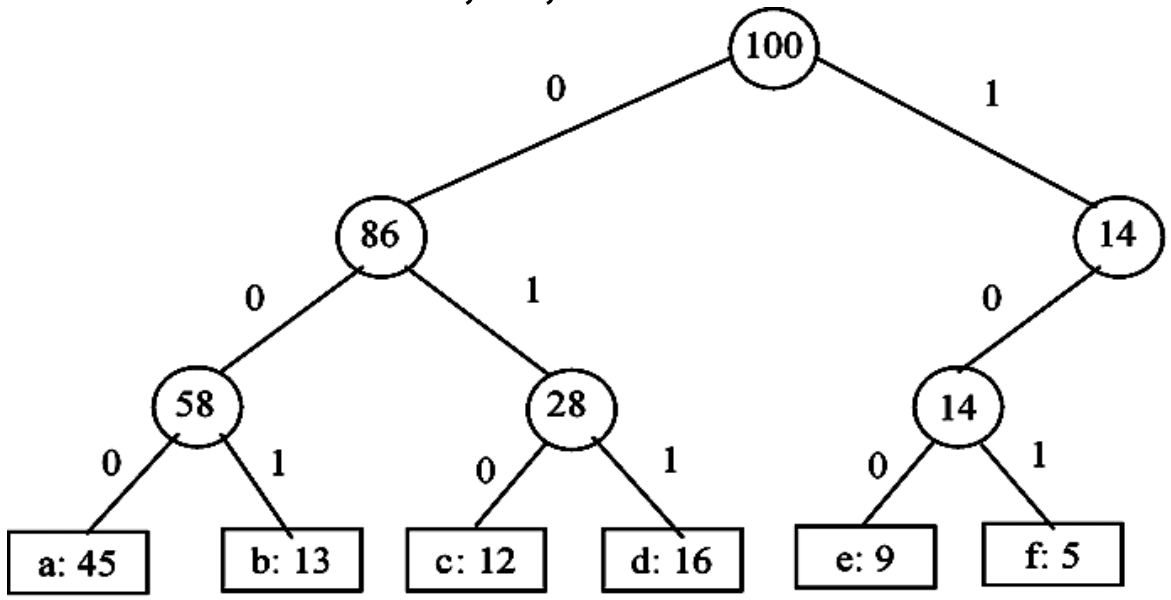
ปัญหา : จะหา prefix code ที่จะใช้หน่วยความจำ (จำนวนบิต) น้อยที่สุดได้อย่างไร

Applications of Binary Tree : Huffman coding

Binary Trees and prefix codes

a) Tree ของ prefix code แบบ fixed-length

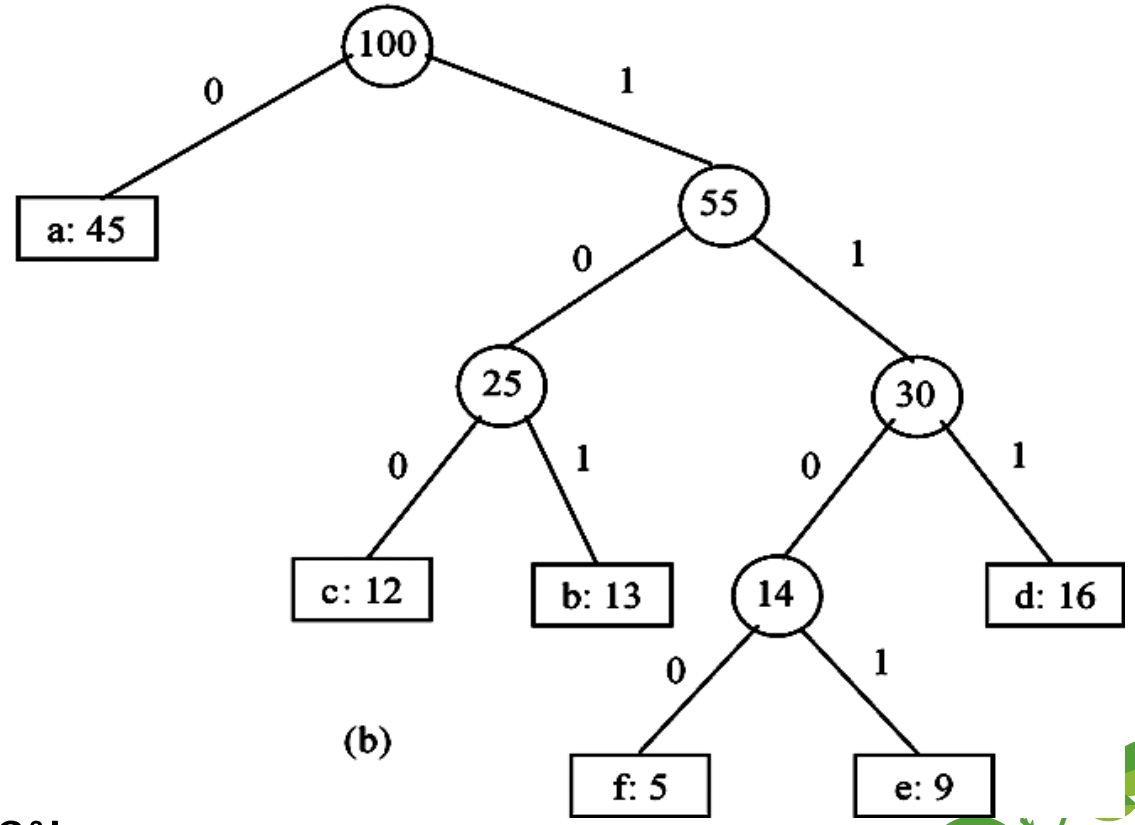
สำหรับรหัส a=000, …, f=100



(a)

b) Tree ของ optimal prefix code

สำหรับรหัส a=0, b=101, …, f=1100



(b)

C = {a=45, b=13, c=12, d=16, e =9, f = 5}

เซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ

Applications of Binary Tree : Huffman coding

Huffman coding ใช้เทคนิค greedy algorithm สำหรับการแก้ปัญหาการหา minimum cost prefix code จะได้ prefix code ที่จะใช้หน่วยความจำน้อยที่สุด เรียกว่า Huffman code

- ✓ **เป็น Full binary tree (internal node มีทั้งโหนดลูกทางซ้ายและขวา)**
- ✓ **จำนวนโหนด leaves จะเท่ากับขนาดของเซต C (จำนวนตัวอักษร)**
  - **แต่ละโหนดจะเก็บค่าตัวอักษร และความถี่ในการปรากฏบนข้อความ**
- ✓ **internal nodes**
  - **มีจำนวนจะเท่ากับ |C| - 1**
  - **เก็บค่าความถี่สะสมของโหนดลูกทั้งสอง**
  - **เส้นทางซ้าย (Left edge) จะแทนด้วยค่าบิต 0**
  - **เส้นทางซ้าย (Right edge) จะแทนด้วยค่าบิต 1**
- **แต่ละ Path จากโหนด root ไปยัง leaf node จะได้ค่ารหัสสำหรับอักขระที่เก็บใน leaf node นั้น**

Applications of Binary Tree : Huffman coding

Huffman coding algorithm

Input: C เป็นเซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ

1. เลือก 2 ตัวอักษร (x และ y) จากเซต C ที่มีความถี่ในการปรากฏบนข้อความน้อยสุด แล้วสร้าง subtree ของ 2 ตัวอักษรนี้ (กำหนดให้ root ของ subtree ที่ได้แทนด้วย z)
2. หาค่าความถี่สะสมของ z , f(z) = f(x) + f(y) แล้วปรับเซต C โดยลบตัวอักษร x และ y แล้วเพิ่ม z ดังนี้

$$C = C \cup \{z\} - \{x, y\} \Rightarrow |C| = |C| - 1$$

ทำซ้ำจนกระทั่ง $|C| = 1$

f(x) คือความถี่ (เปอร์เซ็นต์) ในการปรากฏอักขระ x ในข้อความ (หรือแฟ้มข้อมูล) ที่ต้องการเข้ารหัส

Applications of Binary Tree : Huffman coding

Huffman coding algorithm

Input: เซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ (C[],Freq[])

Output: Binary Huffman tree (Huffman code), L[0]

```
HuffmanCoding(C[], Freq[], n)
1.  for i = 1 to n  // สร้างรายการ node ของรหัสทั้งหมดเก็บใน L
2.      x  = createNewNode(C[i], Freq[i] ) // x.left = x.right = nullptr;
3.      L[i]=x; //L.insert(x)
```

Applications of Binary Tree : Huffman coding

Huffman coding algorithm

```
HuffmanCoding(C[], Freq[], n)
4. for i = 1 to n – 1
    //identify the two least-frequent objects to merge together
5.      x = L.removeMIN()
6.      y = L.removeMIN()
7.      z = createNewNode(nullptr, x.freq + y.freq)
8.      z.left = x ;  z.right =  y;
9.      L.insert(z)
10. return root of the Huffman tree  //L[0] เก็บโหนด root
```

Applications of Binary Tree : Huffman coding

แบบฝึกหัด

Input: เซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ , C

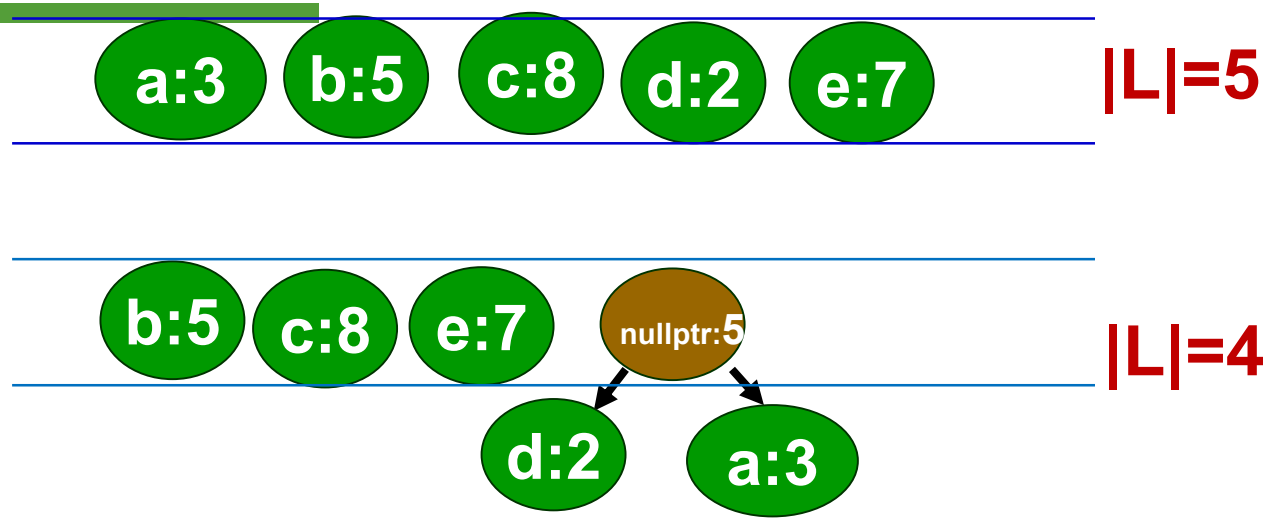
1. Let C = {a=45, b=13, c=12, d=16, e=9, f=5}
2. Let C = {a=20, b=15, c=5, d=15, e=45}
3. Let C = {a=3, b=5, c=8, d=2, e=7}

Output: Huffman codes

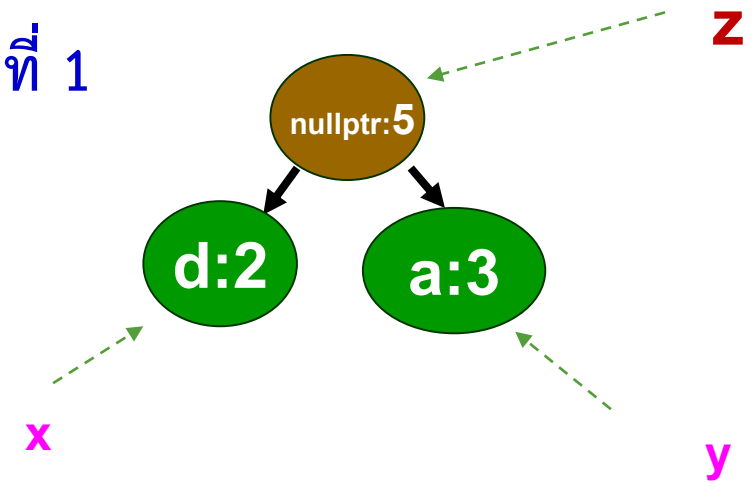
1. a =0,b=101,c=100,d=111,e=1101,f=1100
2. a = 111, b = 101, c = 100, d =110, e = 0
3. a = 011, b = 00 ,c = 10, d = 010, e = 11

Applications of Binary Tree : Huffman coding

// สร้างรายการ node ของรหัสทั้งหมดเก็บใน L



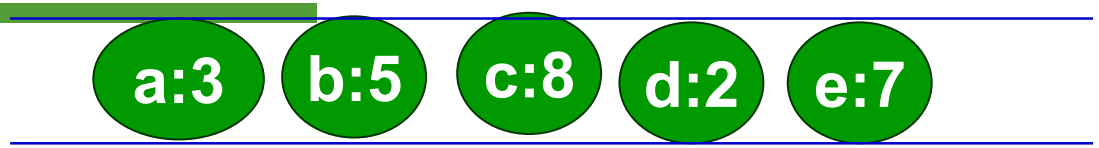
รอบที่ 1




```
x = L.removeMIN()

y = L.removeMIN()

z = createNewNode(nullptr, x.freq + y.freq)

z.left = x ;  z.right =  y

L.insert(z)
```

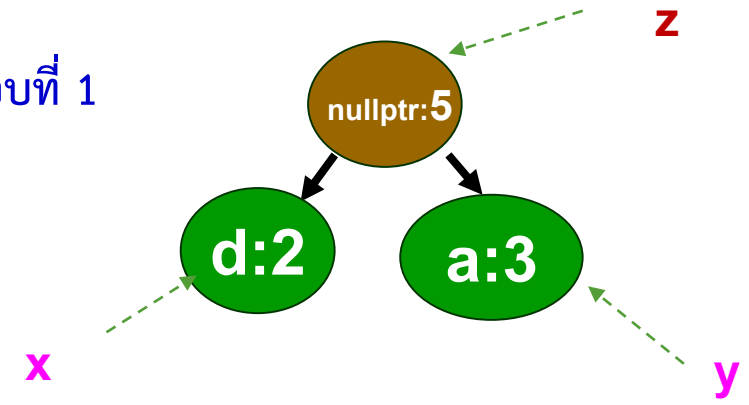
Applications of Binary Tree : Huffman coding

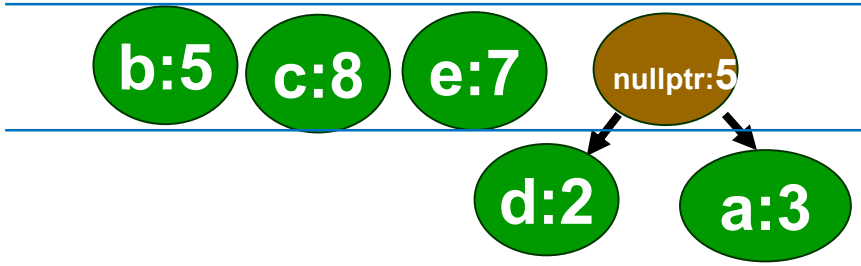
รอบที่ 1



|L|=5

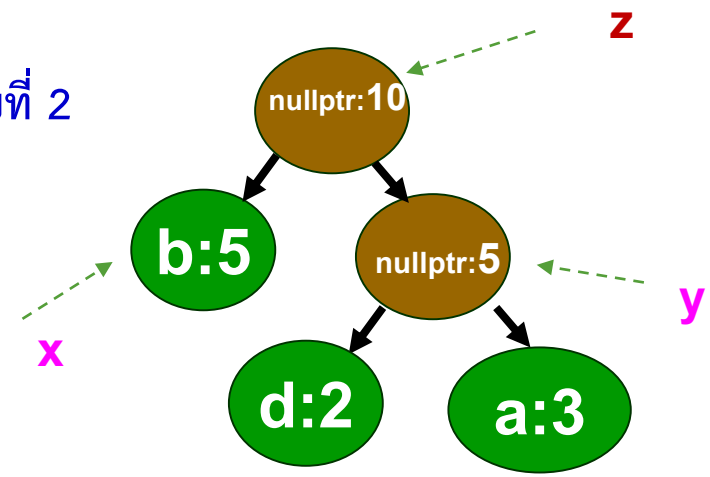
รอบที่ 1

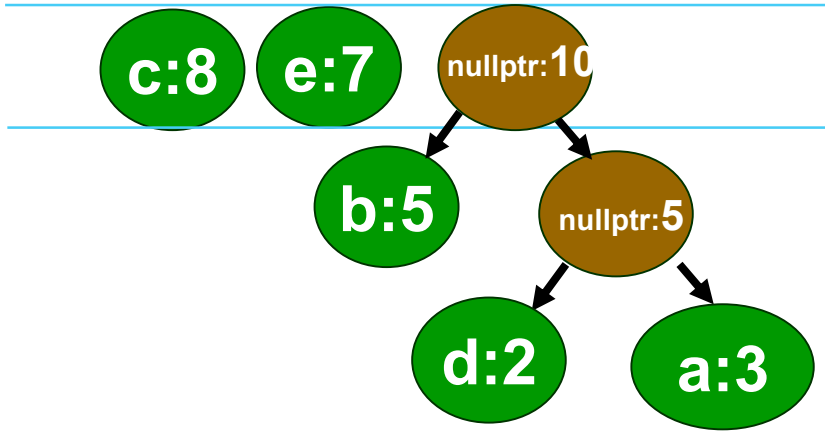




|L|=4

รอบที่ 2



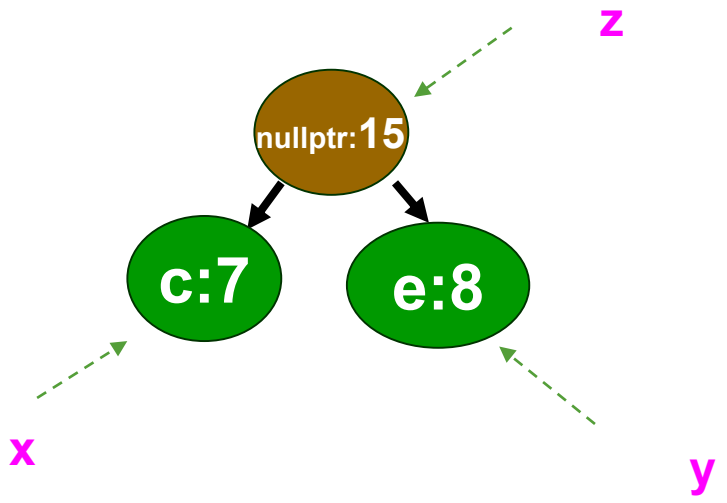


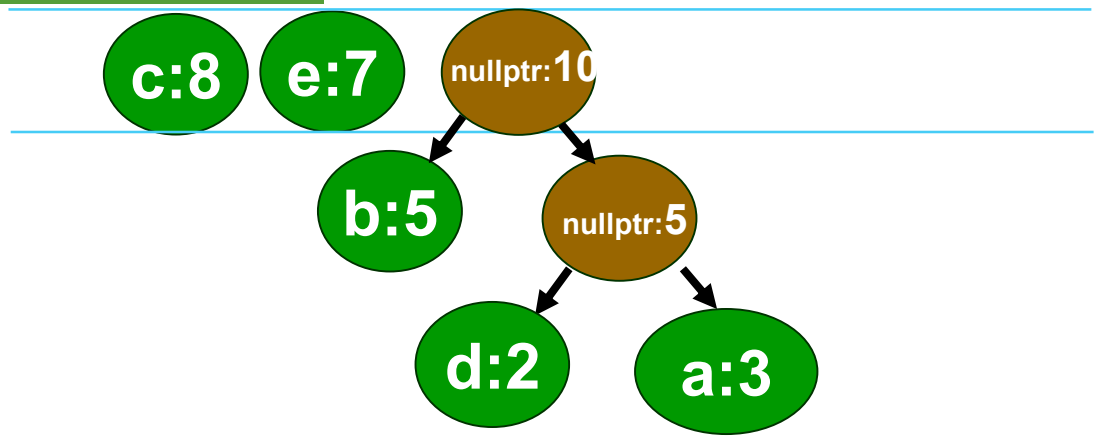
|L|=3

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

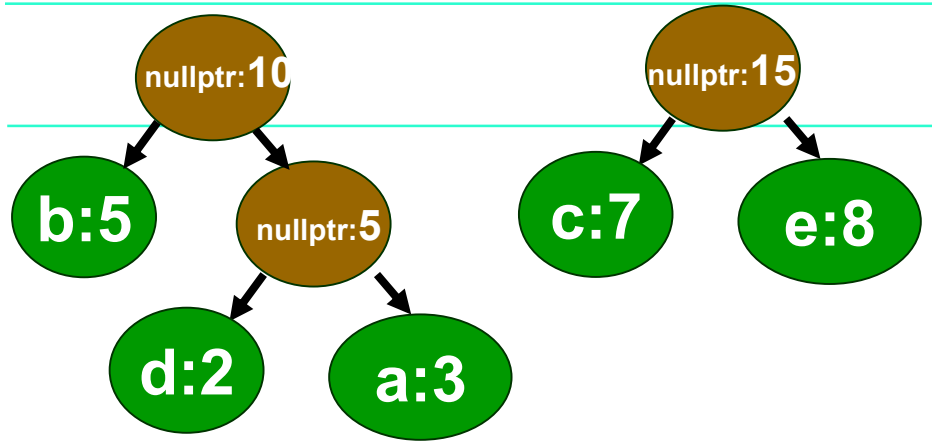
Applications of Binary Tree : Huffman coding

รอบที่ 3





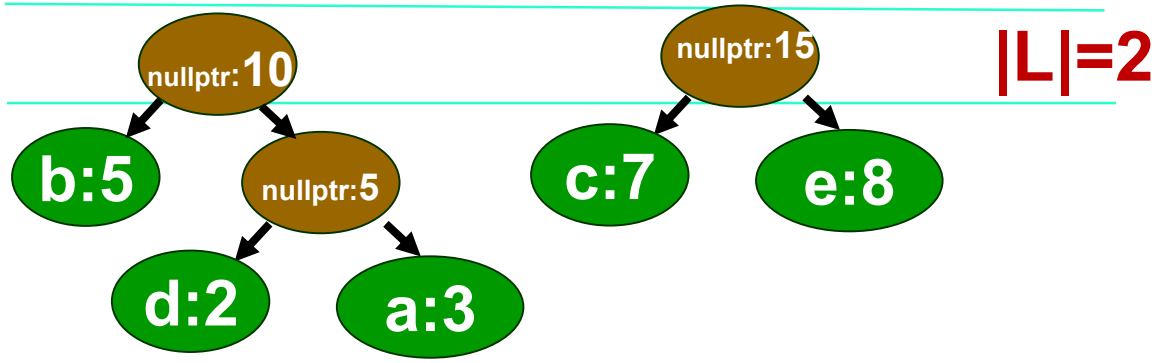
|L|=3



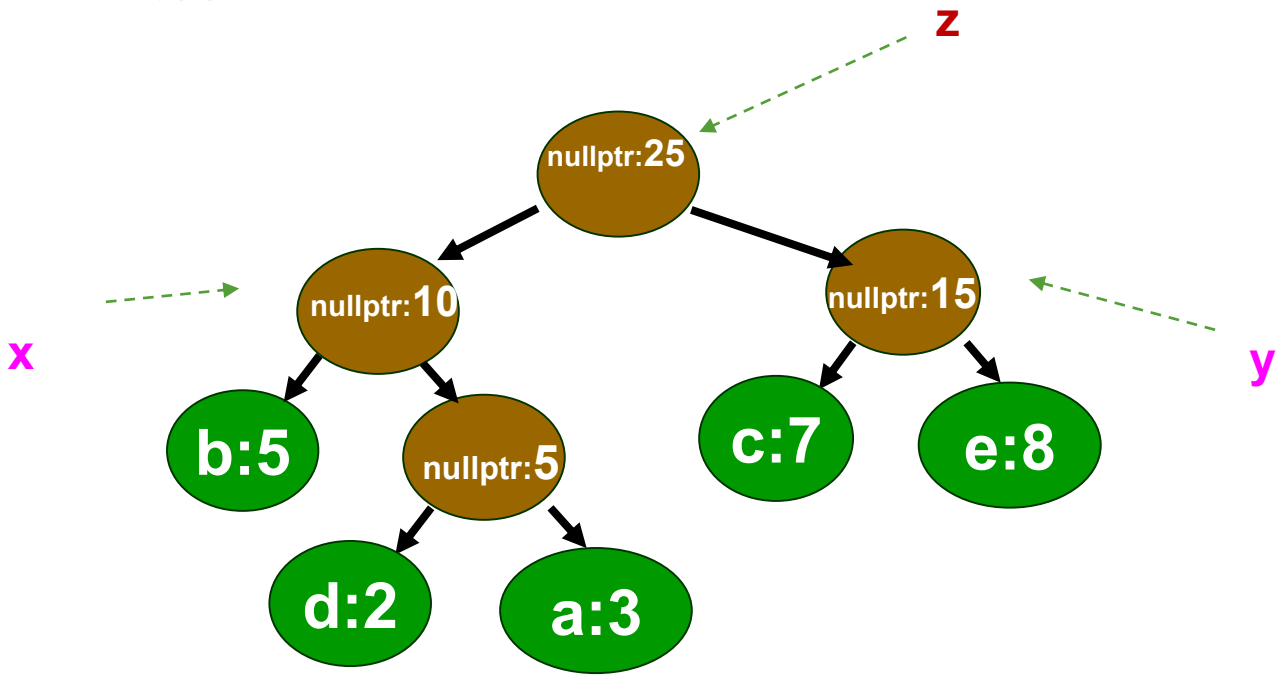
|L|=2

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Applications of Binary Tree : Huffman coding



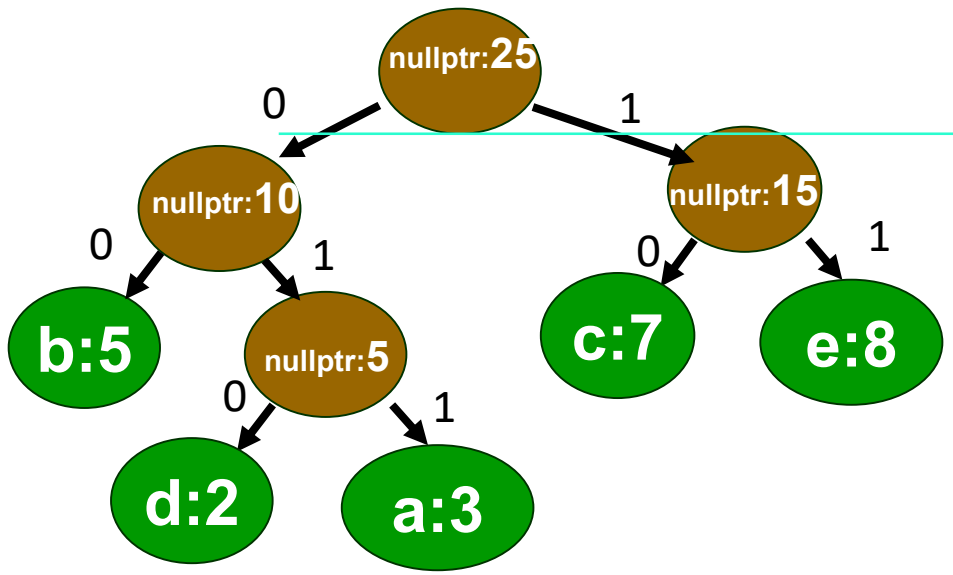
รอบที่ 4



REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Applications of Binary Tree : Huffman coding

- สุดท้ายจะเหลือ node ที่เป็น root ของ Huffman tree



|L|=1

จาก tree จะได้ Huffman codes ดังนี้

(กรณี left node ให้รหัส 0)

a = 011, b = 00 ,c = 10, d = 010, e = 11

ตัวอย่าง เข้ารหัส message ของ abe จะได้ codewords เป็น 0110011

ตัวอย่าง ถอดรหัส codewords ของ 111000011 จะได้ message เป็น ecba

Applications of Binary Tree : Huffman coding

แบบฝึกหัด

Input: เซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ , C

1. Let C = {a=45, b=13, c=12, d=16, e=9, f=5}
2. Let C = {a=20, b=15, c=5, d=15, e=45}
3. Let C = {a=3, b=5, c=8, d=2, e=7}

Output: Huffman codes

1. a =0,b=101,c=100,d=111,e=1101,f=1100
2. a = 111, b = 101, c = 100, d =110, e = 0
3. a = 011, b = 00 ,c = 10, d = 010, e = 11

Input :รับเซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ

Output:แสดงรหัส Huffman codes ของแต่ละตัวอักษร

Input

abcde

3 5 8 2 7

Output

a 011
b 00
c 10
d 010
e 11

Input

abcdef

45 13 12 16 9 5

Output

a 0
b 101
c 100
d 111
e 1101
f 1100

Input

abcde

20 15 5 15 45

Output

a 111
b 101
c 100
d 110
e 0

REFERENCE: DATA STRUCTURE AND ALGORITHMS IN C++, GOODRICH, ET AL.., WILEY, 2011

Input :รับเซตของตัวอักษรและความถี่ในการปรากฏบนข้อความ

Output:แสดงรหัส Huffman codes ของแต่ละตัวอักษร

Input

abcdef

45 13 12 16 9 5

100011001101

Output

a 0

b 101

c 100

d 111

e 1101

f 1100

cafe

Input

abcde

20 15 5 15 45

11000110

Output

a 111

b 101

c 100

d 110

e 0

deed

พัฒนาเพิ่ม รับข้อความ Huffman codes แล้วแสดง message